

Formalizing the Linux eBPF Core ISA: A Mechanized Operational Semantics and Its Real-World Applications

SHENGHAO YUAN^{*†}, Zhejiang University, China

YAZHOU TANG^{*}, Zhejiang University, China

TIANJI CAO, Zhejiang University, China

FREDERIC BESSON, INRIA, France

JEAN-PIERRE TALPIN, INRIA, France

MINGSHUAI CHEN[‡], Zhejiang University, China

This paper presents a mechanized formal semantics for the Linux eBPF instruction set architecture (ISA). We develop a small-step semantics in Rocq that faithfully formalizes all 160 sequential in-kernel instructions of the eBPF ISA. The semantics is fully executable and has been validated against the official Linux eBPF test suite. This extensive testing revealed inconsistencies in our original formalization. Using this semantics, we have designed, implemented, and verified the soundness of the bit-level abstract domain employed by the Linux eBPF verifier. Our semantics also complements the existing Linux eBPF documentation by providing a rigorous formal specification. During the formalization process, we have discovered previously unknown bugs in the Linux eBPF implementation, and developed new verifier optimizations; all of these changes have been upstreamed to the latest version of the Linux kernel.

CCS Concepts: • **Software and its engineering** → **Semantics; Software verification and validation.**

Additional Key Words and Phrases: eBPF, Formal Verification, Mechanized Semantics, Rocq

ACM Reference Format:

Shenghao Yuan, Yazhou Tang, Tianci Cao, Frederic Besson, Jean-Pierre Talpin, and Mingshuai Chen. 2026. Formalizing the Linux eBPF Core ISA: A Mechanized Operational Semantics and Its Real-World Applications. *Proc. ACM Program. Lang.* x, OOPSLA2, Article x (April 2026), 27 pages. <https://doi.org/xx.xxxx/xxxxxxx>

1 Introduction

As a key feature of the Linux kernel, extended Berkeley Packet Filter (eBPF) forms a virtual ISA that, interpreted by a privileged virtual machine (VM), introduces programmability at the kernel level in place of previously fixed kernel subsystems. The eBPF infrastructure of Linux allows to tool the kernel with runtime-triggered services to monitor its security, networking, and resources.

Developers typically write eBPF programs in a restricted dialect of C or Rust and then compile them into eBPF bytecode. As shown in Fig. 1, the Linux eBPF subsystem loads this user-space bytecode into the kernel. The eBPF verifier then employs a combination of abstract interpretation [Cousot and Cousot 1977] and symbolic execution [King 1976] to verify critical properties, e.g., termination and memory safety. Upon successful checking, the verifier also translates the *user-space bytecode* into *in-kernel bytecode*. During this translation, it performs optimizations such as eBPF-specific dead-code elimination. Additionally, it inserts extra instructions and generates

^{*} Authors made an equal contribution.

[†] The author is with the State Key Laboratory of Blockchain and Data Security & Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou, China

[‡] Corresponding author

Authors' Contact Information: Shenghao Yuan, Zhejiang University, Hangzhou, China; Yazhou Tang, Zhejiang University, Hangzhou, China; Tianci Cao, Zhejiang University, Hangzhou, China; Frederic Besson, INRIA, Rennes, France; Jean-Pierre Talpin, INRIA, Rennes, France; Mingshuai Chen, Zhejiang University, Hangzhou, China, m.chen@zju.edu.cn.

non-user-visible instructions (e.g., tail calls) to align with in-kernel execution semantics. Finally, according to user configuration, the subsystem selects an execution engine to run the verified in-kernel bytecode either through an interpreter or a target-specific Just-In-Time (JIT) compiler.

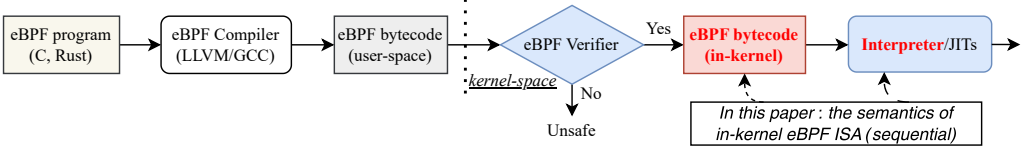


Fig. 1. Linux eBPF VM: Architecture

The C implementation of Linux eBPF is extraordinarily sophisticated and error-prone. For example, its in-kernel verifier contains over 18 thousands of lines of code (KLoC). Statistics [Mohamed et al. 2023] from 2021 to 2023 show that approximately 44.4% of reported eBPF vulnerabilities involved the verifier, including privilege escalation [Google-Inc. 2023; MITRE-Corporation 2016] and denial-of-service attacks [MITRE-Corporation 2021]. To reduce these issues, researchers have applied formal techniques to verify both the some parts of the eBPF verifier [Gershuni et al. 2019; Hamza et al. 2025; Sun and Su 2025; Vishwanathan et al. 2022, 2023; Wu et al. 2025] and JIT compilers correctness [Nelson et al. 2020; Van Geffen et al. 2020].

This paper presents an initial step toward a far more ambitious goal: verifying the functional correctness of the entire Linux eBPF verifier. We observe that despite numerous formal efforts targeting Linux eBPF, the eBPF ISA itself still lacks a formal specification—particularly for the precise semantics of in-kernel bytecode. The latest eBPF Systematization of Knowledge paper [Huang et al. 2025] highlights this issue as a root cause of memory safety concerns: “*The lack of formal specification leads to an unsound ... verification process*”. Therefore, a mechanized and faithful formalization of Linux eBPF semantics is essential for achieving our long-term goal.

Previous work on formal semantics for eBPF has either mechanized a restricted subset of the Linux eBPF ISA—lacking consistency guarantees [Van Geffen et al. 2020] or introducing deliberate adaptations [Priya et al. 2025], or focused on eBPF variants [Yuan et al. 2022, 2025; Zandberg et al. 2022] for other platforms (e.g., RIOT-OS [Zandberg and Baccelli 2020] and Solana [Solana-labs 2018]). These variants diverge from the Linux eBPF baseline. In contrast, this paper develops a formal semantics for the in-kernel Linux eBPF ISA. We present a mechanized semantics that covers all sequential instructions of the Linux eBPF ISA. Our semantics provides a precise specification that serves as a formal foundation for future eBPF standardization. We have also applied it to verify and enhance the bit-level abstraction of the existing in-kernel verifier. We exclude atomic instructions from the current eBPF ISA and discuss this limitation and its research implications. We mechanized all formalizations and proofs in the Rocq proof assistant [Bertot and Castran 2010], which allows us to reuse verified components from the CompCert project [Leroy and Blazy 2008].

1.1 Key Challenges

To guarantee that our formalization faithfully represents the in-kernel Linux eBPF ISA, we must overcome several additional challenges.

C1: Imprecise and Incomplete Documentation. Existing documentation for the Linux eBPF ISA provides only textual descriptions. It lacks a precise and complete formal semantics, which introduces ambiguity and omits critical details. For example, a recent draft submitted to the IETF BPF Working Group [Thaler 2024] specifies the 64-bit unsigned modulo instruction as “dst % (u32)imm” when the 32-bit signed imm is nonzero. However, the actual kernel implementation

follows “dst % (u64) imm”. Such discrepancies can lead to inconsistencies between documentation and formalization. Moreover, the Linux eBPF documentation [Linux-Maintainers 2026] does not detail the eBPF call mechanism (e.g., how the stack pointer changes during execution). It also omits certain instructions, such as tail-call. These gaps hinder non-kernel developers and verification engineers. More importantly, they complicate any attempt to construct a faithful formal semantics.

C2: Semantic Complexity of Linux eBPF. The semantics of eBPF instructions depends on two core components: the interpreter and the verifier. Both components affect runtime behavior and therefore must be reflected in the formal model. First, the interpreter implements eBPF instructions through Linux-specific APIs. For example, the signed 64-bit division instruction calls `div64_s64` from the Linux `math64` library. To model this instruction faithfully, we must analyze all corner cases, such as division by negative one. These dependencies increase the complexity of the formal semantics. Second, the verifier changes program behavior through post-analysis rewrites. Its function `do_misc_fixups` inserts additional instructions to enforce consistent semantics. In particular, it ensures that division by zero always returns zero. Therefore, the verifier not only checks programs but also alters their effective behavior.

C3: Maintaining Formal Semantics under Kernel Evolution. The Linux kernel evolves rapidly, and eBPF-related components change accordingly. These updates directly affect the semantics. For example, Linux v5.14 introduced new eBPF map pointer behaviors that modify the semantics of LDDW (load 64-bit value). Linux v6.6 added new eBPF instructions like signed division operations (BPF_SDIV). Such changes can render existing documentation and prior formal work outdated.

The absence of a precise formal specification further increases the risk of subtle errors during these updates. Indeed, during our formalization process, we identified a bug related to the introduction of BPF_SDIV in eBPF (details in Section 3.5). This experience demonstrates that semantic changes require rigorous validation. Consequently, only a continuous synchronization process can ensure our formalization remains faithful to the latest Linux implementation over time.

1.2 Contributions

In this paper, we address the above challenges by developing a comprehensive formal semantics of the Linux eBPF in-kernel ISA with a faithfulness guarantee, as shown in Fig. 2. We further demonstrate its practical value through real-world applications in Linux eBPF and related systems. In summary, we make the following contributions:

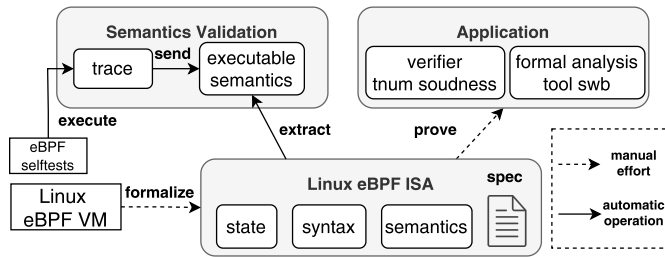


Fig. 2. Methodological overview and component relationships

Mechanization. We define an operational small-step semantics for the Linux eBPF ISA and mechanize it in Rocq (to address C1). The formal model covers all 160 in-kernel eBPF sequential instructions, including 148 user-available instructions and 12 internal opcodes. Using this semantics,

we strengthen the existing eBPF standardization effort by providing a rigorous formal specification with synchronization (to address *C1,C3*). During the formalization, we have uncovered two real-world bugs in the Linux eBPF subsystem — confirmed by the kernel maintainers.

Consistency. To validate our semantic model (to address *C2*), we extract an executable interpreter from the mechanized semantics and test its faithfulness against the original C implementation. On the official Linux (v7.0-rc3) benchmarks—more than 1700 test cases—we obtain execution traces equivalent to those emitted by the Linux eBPF interpreter. Furthermore, this workflow successfully exposed several deeply hidden inconsistencies in our initial Rocq model.

Usability. We illustrate the potential (also for *C2*) of our semantics for formal analyses of modeling and proving the soundness of the abstraction domain (named *trnum*) in the Linux eBPF verifier and a formal analysis tool *swb* [Fan et al. 2025]. Our work has provided the precise of the in-kernel verifier and also uncovered several soundness and optimization issues—confirmed by the *swb* authors.

Our work has limitations that we state here. First, we do not formalize the in-kernel eBPF atomic instructions that relate to the eBPF concurrency memory model discussed in Section 6. Instead, we focus on the remaining in-kernel eBPF instructions that execute in a thread-local environment which aligns with our long-term goal of formally verifying the entire in-kernel verifier. As a result, extending the formalization to cover atomic instructions remains an important direction for future work. Second, we do not yet prove the correctness of the entire Linux eBPF verifier in Rocq. Nevertheless, the current effort allows us to gain familiarity with the verifier’s complex design and to restructure the codebase from a formal semantics perspective. While these are limitations we expect to address in follow up works, we believe the challenges addressed in this paper mark a significant step forward in the state of the art.

Paper Structure. The rest of the paper is organized as follows. Section 2 introduces necessary background on eBPF family and CompCert. Section 3 presents the mechanized semantics of the Linux eBPF ISA. Section 4 proposes a new workflow to validate the semantics. Section 5 details the semantics application for the Linux eBPF verifier. Section 6 discusses the Linux eBPF concurrency model. Section 7 introduces related research, Section 8 summarizes, and the last section provides the data-availability statement.

2 Background

This section introduces the essential concepts of Linux eBPF and CompCert.

2.1 Linux eBPF

Berkeley Packet Filters (BPF), also known as cBPF [McCanne and Jacobson 1993], provided Unix-BSD systems with network packet filtering capabilities. Linux kernel v3.18 extended this design and introduced extended BPF (eBPF). eBPF enables application developers to execute sandboxed programs inside the kernel for additional capabilities like observability or security.

eBPF ISA. The eBPF instruction set architecture derives from a 64-bit RISC design. It defines 10 general-purpose registers and one read-only stack pointer register. eBPF supports two instruction encodings, as shown in Fig. 3. A basic 64-bit eBPF instruction consists of the following fields, ordered from the least significant bit (lsb) to the most significant bit (msb): an 8-bit opcode, two 4-bit register fields (destination and source), a 16-bit signed offset, and a 32-bit signed immediate. The placement of the register fields depends on the hardware endianness. In addition, eBPF defines a wide 128-bit instruction encoding to support 64-bit immediates. The first 64 bits follow the basic instruction format. The second 64 bits are zero except for the immediate field.

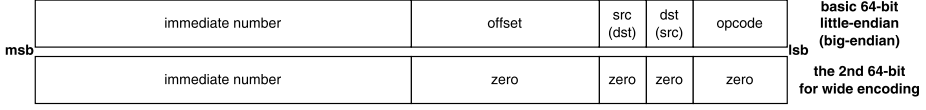


Fig. 3. Linux eBPF instruction encodings

As of Linux v7.0-rc3, the eBPF ISA defines 172 opcodes. Among them, 160 are available to users and cover ALU operations, memory accesses, jumps, and 12 atomic operations. The remaining 12 opcodes are not user-accessible and appear only *e.g.*, in the interpreter. These internal opcodes serve various purposes, *e.g.*, *tail-call* to reuse of the local eBPF stack frame.

Each eBPF instruction corresponds to an opcode macro in the kernel. The 8-bit opcode field divides into three parts. The high 4 bits represent the operation code, *e.g.*, BPF_AND (0x50) for logical and. The low 3 bits indicate the instruction class, *e.g.*, BPF_JMP (0x05) for 64-bit branch operations, BPF_ALU (0x04) for 32-bit arithmetic operations and BPF_ALU64 (0x07) for 64-bit ones. The remaining 1 bit specifies the source operand type: 0 for an immediate value (kernel macro BPF_K = 0x00) and 1 for a source register (kernel macro BPF_X = 0x08). For instance, BPF_ALU | BPF_AND | BPF_K (= 0x5c) is the opcode for a 32-bit eBPF and-operation with an immediate.

The user-space eBPF bytecode are translated into in-kernel bytecode by the verifier, we illustrate the difference between the user-space and in-kernel ISA with two representative examples:

- *map-access*¹: A user program can request access to a map using the pseudo-instruction BPF_LD_IMM64_RAW(R1, BPF_PSEUDO_MAP_IDX, 0). The macro BPF_PSEUDO_MAP_IDX (value 5, stored in the src field) indicates that the instruction refers to a map by index (here index 0) and the opcode 0x18, *i.e.*, load double-words LDDW, stores the 64-bit resulting pointer in the destination register R1. The verifier uses the function `resolve_pseudo_ldimm64` to resolve this symbolic reference to the concrete kernel map address, *e.g.*, 0xffff889912345678. Consequently, the in-kernel binary encoding becomes [0x18; 0x51; 0x00; 0x00; 0xff; 0xff; 0x88; 0x99; 0x00; 0x00; 0x00; 0x00; 0x12; 0x34; 0x56; 0x78].
- *tail-call*: To reuse the current eBPF local stack, the in-kernel verifier translates specific user-space eBPF call instructions into tail-call instructions via the function `do_misc_fixups`. The macro opcode for a call is BPF_JMP | BPF_CALL (0x85); it translates this to BPF_JMP | BPF_TAIL_CALL (0xf5). These two opcodes correspond to entirely different behaviors.

eBPF Hooks. Each eBPF program operates in an event-driven manner: it executes when the kernel or an application reaches a specific hook point. These hooks are either predefined or user-created. Predefined hooks include system calls, kernel tracepoints, and network events. User-created hooks, such as kernel probes (kprobes) [Goswami 2005] or user probes (uprobes) [Dronamraju 2012], allow eBPF programs to attach to almost any location in kernel or user applications. One can also extend the kernel with custom hooks for specialized purposes.

2.2 CompCert

CompCert is a C compiler written and verified in Rocq. It translates C programs to assembly for several architectures, including x86, ARM, and RISC-V. This section only presents the parts relevant to this paper. For a complete description, we refer the reader to [Leroy et al. 2012].

Configuration. CompCert provides an architecture configuration module `Archi`. It declares two key parameters: i) `Archi.big_endian`: `bool` indicates the endianness; it is `true` for big-endian

¹eBPF uses maps, which are shared key-value data structures, to exchange data between kernel and user space.

and false for little-endian. ii) `Archi.ptr64`: `bool` specifies the pointer width; it is true for 64-bit architectures and false for 32-bit ones.

Value. A value v in CompCert can be an undefined value $\perp$, a 64-bit integer `Vlong` i , a 32-bit integer `Vint` i , a pointer `Vptr` $blk\ ofs$ (containing a block identifier blk and an offset ofs), or other cases. The null pointer definition depends on the architecture:

$$\begin{aligned} \text{val } \ni v &::= \perp \mid \text{Vlong } i \mid \text{Vint } i \mid \dots \mid \text{Vptr } blk\ ofs \\ \text{Vnullptr} &\stackrel{\text{def}}{=} \text{if } \text{Archi.ptr64} \text{ then } \text{Vlong } 0 \text{ else } \text{Vint } 0 \end{aligned}$$

Memory. CompCert's memory model comprises separate arrays. Each array has a fixed size, determined at allocation time, and is identified by an uninterpreted block $b \in \text{block}$. The memory interface provides several core operations:

- `Mem.alloc` (M, lo, hi) = (b, M'): Allocates a fresh block with bounds $[lo, hi]$ and initializes it with undefined values. It returns the new block identifier b and the new memory state M' .
- `Mem.load` (mc, M, b, ofs) = $[v]$: Reads a value v of chunk mc from memory M at block b starting at offset ofs . The variant `Mem.loadv` (mc, M, ptr) takes a pointer value ptr instead of separate block and offset arguments.
- `Mem.store` (mc, M, b, ofs, v) = $[M']$: Writes the value v into chunk mc of block b at offset ofs . It returns the updated memory M' on success. Similarly, `Mem.storev` (mc, M, ptr, v) uses a pointer value ptr .
- `Mem.free` (M, b, lo, hi) = $[M']$: Frees the interval $[lo, hi]$ of block b . On success, it returns M' , in which this interval is invalidated and no longer accessible.

The memory chunk $mc ::= \text{Mint8} \mid \dots \mid \text{Mint64}$ in load and store operations specifies the size and interpretation of the accessed data, e.g., `Mint8` specifies a 8-bit value. CompCert assumes an infinite memory, so `Mem.alloc` always succeeds. However, `Mem.load` and `Mem.store` can fail due to an invalid block b , an out-of-bounds offset ofs , or insufficient permissions. The `Mem.free` operation may also fail if the specified range lacks freeable permissions. Therefore, these operations return option types: $[v]$ (i.e., *Some* v) indicates success with result v , while $\emptyset$ (i.e., *None*) denotes failure.

3 Operational Semantics

This section introduces the operational semantics of the Linux eBPF sequential ISA. Following the standard small-step semantics style, each step has the form $s \xrightarrow{\text{ins}} s'$ which performs one step execution of the eBPF instruction ins from the initial state s to the new state s' .

Notably, although our semantics is mechanized in Rocq and fully leverages the existing infrastructure (e.g., the memory model) of the CompCert project, one could also implement it using other frameworks. Alternative options include the Sail language [Armstrong et al. 2019] or proof assistants such as Isabelle/HOL [Nipkow et al. 2002] and Lean [de Moura et al. 2015].

3.1 Program State

The normal program state is a quadruple $\mathcal{S} ::= \langle \mathcal{R}, \mathcal{M}, \mathcal{STK}, cnt \rangle$, consisting of

- the register state $\mathcal{R}$ is a mapping from the eBPF registers to CompCert value: ' $\mathcal{R} : r \rightarrow \text{val}$ ' where r includes $r_1 - r_9$ (general-purpose), r_{10} (eBPF local stack pointer), r_{11} (kernel hidden register used by verifier and JITs, its kernel macro is `AX`), and pc (program counter);
- the CompCert memory model $\mathcal{M}$;
- the abstract eBPF call stack ' $\mathcal{STK} : \text{list callInfo}$ ' stores the call information, where callInfo is a 6-tuple $\langle v_6, v_7, v_8, v_9, v_{10}, ra \rangle$ records the value of four caller-save registers, the old eBPF stack pointer, and the return address. The list size represents the internal call depth;

- *cnt* records the amount of eBPF tail calls.

Initial and Final States. We define two distinguished eBPF states as follows.

- **Initial State:** Given three memory blocks—the program context *ctx*, the code region *pc_blk*, and the eBPF local stack *stk*—the initial state satisfies the following conditions:
 - (1) all registers are undefined except for *pc* (program counter), which points to the first instruction in the eBPF code; *r₁₀*, which points to the top of the local stack; and *r₁*, which points to the start address of the input context;
 - (2) the eBPF call stack is empty and the initial tail call counter is zero.

Inductive $\text{initial_state}(ctx\ pc_blk\ stk : block) : \mathcal{S} \rightarrow Prop := \forall \mathcal{R}, \mathcal{M}.$

$(\mathcal{R} = \lambda r. \text{match } r \text{ with}$

$| pc \Rightarrow \text{Vptr}(pc_blk, 0) \mid r_{10} \Rightarrow \text{Vptr}(stk, 512) \mid r_1 \Rightarrow \text{Vptr}(ctx, 0) \mid _ \Rightarrow \perp) \rightarrow$
 $\text{initial_state } ctx\ pc_blk\ stk\ (\mathcal{R}, \mathcal{M}, [], 0)$

Inductive $\text{final_state}(stk : block) : \mathcal{S} \rightarrow val \rightarrow Prop := \forall \mathcal{R}, \mathcal{M}, res, n.$

$\mathcal{R}(pc) = \text{Vnullptr} \wedge \mathcal{R}(r_0) = res \wedge \mathcal{R}(r_{10}) = \text{Vptr}(stk, 512) \wedge \text{is_ptr_or_int64 } res \rightarrow$
 $\text{final_state } stk\ (\mathcal{R}, \mathcal{M}, [], n)\ res$

- **Final State:** Similar to the initial state, e.g., *r₁₀* still points to the top of the same local stack. The final state differs primarily in that the *pc* register points to a null pointer, and the return value register *r₀* holds either a pointer or a 64-bit integer value.

3.2 eBPF Syntax

The original eBPF instructions are defined as C structs, with opcodes constructed from kernel macros. For example, the 64-bit eBPF addition opcode that uses a register as source operand is written as ‘BPF_ALU64 | BPF_ADD | BPF_X’ in the kernel. However, to better align with Rocq’s inductive definition style and to simplify the representation, we adopt a more structured formalization. Fig. 4 presents the core eBPF ISA, organized into three instruction categories.

- **ALU** includes 32-bit and 64-bit arithmetic and logic operations. These comprise unary operations such as NEG, binary arithmetic operations (e.g., add), logic operations (e.g., and), and shift operations (e.g., left shift lsh), as well as signed and unsigned division and modulo. Additionally, this category contains specialized instructions, including platform-dependent byte swaps (e.g., little-endian END leN and big-endian END beN), platform-independent byte swaps (e.g., END swN), and various move instructions that are either zero-extended (such as MOVI32 and MOVI64) or sign-extended (e.g., MOV mov64sx8).
- **MEM** provides standard memory access operations: load (LD) and store (ST) with 1-, 2-, 4-, or 8-byte sizes. For loading a 64-bit immediate value into a register, eBPF supplies the LDDW instruction, which uses the format LDDW *pty* *r_d* *lo₃₂* *hi₃₂*. The *const* constructor denotes that the 64-bit immediate value is an integer; the other constructors represent various pointers.
- **JUMP** contains conditional jump instructions (JMP) in both 32-bit and 64-bit forms, along with unconditional jumps. The 64-bit unconditional jump JA uses a 16-bit *ofs* field as a short-jump offset, whereas the 32-bit version JA32 uses a 32-bit *imm* field for a long-jump offset. This category also includes three call instructions (one internal and two external) and the return instruction EXIT.

Several eBPF instructions are not directly available to users: TailCall enables partitioning an eBPF program into multiple parts. ST_NoSpec serves for mitigating Speculative Store Bypass. LDSX

(Register)	r	::=	$r_0 \mid r_1 \mid r_2 \mid r_3 \mid r_4 \mid r_5 \mid r_6 \mid r_7 \mid r_8 \mid r_9 \mid r_{10} \mid r_{11} \mid pc$
(ALUOp)	aop	::=	$add \mid sub \mid mul \mid or \mid and \mid xor \mid lsh \mid rsh \mid arsh \mid$ $udiv \mid sdiv \mid umod \mid smod \mid$
(Move)	xop	::=	$mov32sx8 \mid mov32sx16 \mid mov32sx32 \mid$ $mov64sx8 \mid mov64sx16 \mid mov64sx32 \mid mov64sx64$
(Endian)	eop	::=	$le16 \mid le32 \mid le64 \mid be16 \mid be32 \mid be64 \mid sw16 \mid sw32 \mid sw64$
(Pseudo)	pty	::=	$const \mid map_fd \mid map_id \mid map_v \mid map_idv \mid btf_id \mid fptr$
(Condition)	cop	::=	$eq \mid neq \mid lt \mid slt \mid le \mid sle \mid ge \mid sge \mid gt \mid sgt \mid set$
(SecondOp)	sop	::=	$r \mid imm$
(Instruction)	ins	::=	$ALU32 \ aop \ r_d \ sop \mid ALU64 \ aop \ r_d \ sop \mid NEG32 \ r_d \mid NEG64 \ r_d \mid$ $(*ALU*) \ END \ eop \ r_d \mid MOV \ xop \ r_d \ r_s \mid MOVI32 \ r_d \ imm \mid MOVI64 \ r_d \ imm \mid$ $(*MEM*) \ LDDW \ pty \ r_d \ imm \ imm \mid LD \ mc \ r_d \ r_s \ ofs \mid ST \ mc \ r_d \ sop \ ofs \mid$ $(*JUMP*) \ JA \ ofs \mid JA32 \ imm \mid JMP \ cop \ r_d \ sop \ ofs \mid JMP32 \ cop \ r_d \ sop \ ofs \mid$ $CallInternal \ imm \mid CallID \ imm \mid CallBTF \ imm \mid EXIT$ $TailCall \mid ST_NoSpec \mid LDSX \ mc \ r_d \ r_s \ ofs \mid$ $LD_PROBE \ mc \ r_d \ r_s \ ofs \mid LDSX_PROBE \ mc \ r_d \ r_s \ ofs$

Fig. 4. The syntax of Linux eBPF core ISA

performs a signed-extension load. Finally, LD_PROBE (LDSX_PROBE) implements probe-able memory load (with signed-extension) operations.

3.3 eBPF Semantics

Four static information are necessary for the eBPF state transition:

- ‘code: list int64’ represents the (main) eBPF program binary code;
- ‘jmp_tbl: list code’ represents a jump table including several eBPF binary code snippets;
- ‘max_call: nat’ represents the max number of entries in a jump table;
- ‘l_addr: list addr_region’ records the accessible memory region (e.g., local eBPF stack), each region is a triple $\langle base_blk, base_addr, size_blk \rangle$ including the abstract CompCert memory block, the corresponding 64-bit concrete address (int64), and the block’s capacity.

Fig. 5, Fig. 6, and Fig. 7 gather the transition rules for all instructions formalized in this paper, that we explain below. The following notations are used throughout these rules:

- $[[x]]_t$ denotes the value x cast to type t , $[[x]]$ is shorthand for $[[x]]_{u64}$. If x is a register, this notation returns $\mathcal{R}(r)$.
- op_t denotes the operation corresponding to type t , for example, $+_{u64}$ (abbreviated $+_{64}$) represents 64-bit unsigned addition.
- $x.y$ accesses field y of x , while $x[y \leftarrow z]$ updates $x.y$ to z .
- $(uN) \ v$ zero-extends the integer value v to N bits, if v is a pointer, this operation leaves it unchanged. Similarly, $(sN) \ v$ performs a signed-extension to N bits.
- $pc^{+n} \stackrel{\text{def}}{=} pc \leftarrow [[pc]] + n$ updates the eBPF pc register by adding offset n . Since each pc value points to a 64-bit instruction, pc^{+n} effectively skips n instructions. Note that n may be negative, allowing backward jumps.

$$\begin{array}{c}
\frac{ins = \text{ALU32 } aop \ r_d \ sop \quad aop32(r_d, sop) = v}{S \xrightarrow{ins} S[r_d \leftarrow (u64) \ v, \ pc^{+1}]} \quad \frac{ins = \text{ALU64 } aop \ r_d \ sop \quad aop64(r_d, sop) = v}{S \xrightarrow{ins} S[r_d \leftarrow v, \ pc^{+1}]} \\
\\
\frac{ins = \text{NEG32 } r_d \quad -_{32} \llbracket r_d \rrbracket_{u32} = v}{S \xrightarrow{ins} S[r_d \leftarrow (u64) \ v, \ pc^{+1}]} \quad \frac{ins = \text{NEG64 } r_d \quad -_{64} \llbracket r_d \rrbracket = v}{S \xrightarrow{ins} S[r_d \leftarrow v, \ pc^{+1}]} \quad \frac{ins = \text{END } eop \ r_d \quad eop(r_d) = v}{S \xrightarrow{ins} S[r_d \leftarrow v, \ pc^{+1}]} \\
\\
\frac{ins = \text{SX } xop \ r_d \ r_s \quad \text{signExt}(xop, r_s) = v}{S \xrightarrow{ins} S[r_d \leftarrow (u64) \ v, \ pc^{+1}]} \quad \frac{ins = \text{MOVI32 } r_d \ imm \quad \llbracket imm \rrbracket_{u32} = v}{S \xrightarrow{ins} S[r_d \leftarrow (u64) \ v, \ pc^{+1}]} \quad \frac{ins = \text{MOVI64 } r_d \ imm \quad \llbracket imm \rrbracket_{s64} = v}{S \xrightarrow{ins} S[r_d \leftarrow (u64) \ v, \ pc^{+1}]}
\end{array}$$

Fig. 5. Linux eBPF: **ALU** semantics

3.3.1 Semantics: ALU. The Linux eBPF ALU instructions include both 64-bit and 32-bit variants, as shown in Fig. 5. A 64-bit instruction performs its operation and stores the result in the destination register. A 32-bit instruction, such as NEG32 or MOVI32, performs the corresponding 32-bit operation on the lower 32 bits of the source (either a sub-register or an immediate value), then zero-extends the result to 64 bits and writes it to the destination register.

Most eBPF ALU instructions follow standard unsigned arithmetic, similar to other architectures like ARM or RISC-V. However, eBPF defines specific semantics for the following cases:

- **Shift instructions:** Operations such as ALU64 arsh and ALU32 arsh explicitly mask the arithmetic right shift amount stored in a register. For 64-bit shifts, the mask is 63; for 32-bit shifts, the mask is 31. This masking prevents undefined shift behavior.
- **Unsigned division and modulo:** In case of division by zero, eBPF defines that udiv returns zero and umod returns the original value of the destination register. This behavior is formalized in the following aopN definition where $N \in \{32, 64\}$.
- **Signed division and modulo:** For division by zero, signed operations behave the same as their unsigned counterparts. Additionally, eBPF handles a special overflow case: when the divisor is -1 and the dividend is the signed minimum value (i.e., $s32_min = -2^{31}$ for 32-bit or $s64_min = -2^{63}$ for 64-bit), the result of sdiv is the signed minimum value, and the result of smod is zero. In all other cases, signed operations follow standard signed arithmetic.

$$aopN(r_d, sop) \stackrel{\text{def}}{=} \begin{cases} \llbracket r_d \rrbracket_{uN} +_N \llbracket sop \rrbracket_{uN} & , \text{ if } aop = add \\ \dots & \\ \llbracket r_d \rrbracket_{sN} \gg_{sN} (\llbracket sop \rrbracket_{uN} \&_N (N-1)) & , \text{ if } aop = arsh \\ \llbracket sop \rrbracket_{uN} = 0 ? 0 : \llbracket r_d \rrbracket_{uN} /_{uN} \llbracket sop \rrbracket_{uN} & , \text{ if } aop = udiv \\ \llbracket sop \rrbracket_{sN} = 0 ? 0 : (\llbracket sop \rrbracket_{sN} = -1 \&\& \\ \quad \llbracket r_d \rrbracket_{sN} = sN_min ? sN_min : \llbracket r_d \rrbracket_{sN} /_{sN} \llbracket sop \rrbracket_{sN}) & , \text{ if } aop = sdiv \\ \llbracket sop \rrbracket_{uN} = 0 ? \llbracket r_d \rrbracket_{uN} : \llbracket r_d \rrbracket_{uN} \%_{uN} \llbracket sop \rrbracket_{uN} & , \text{ if } aop = umod \\ \llbracket sop \rrbracket_{sN} = 0 ? \llbracket r_d \rrbracket_{sN} : ((\llbracket sop \rrbracket_{sN} = -1 \&\& ? \\ \quad \llbracket r_d \rrbracket_{sN} = sN_min ? 0 : \llbracket r_d \rrbracket_{sN} \%_{sN} (\llbracket sop \rrbracket_{sN})) & , \text{ if } aop = smod \end{cases}$$

The eBPF ISA includes two distinct classes of remainder ALU instructions: endianness operations and zero/sign-extension operations.

eBPF endian operations. eBPF introduces three byte-swap instructions to adapt to different host architectures, particularly when processing network data. These instructions operate as follows:

- **leN:** If the host uses big-endian byte order, the instruction swaps the bytes in the destination register; otherwise, it applies zero-extension to the register.

- **beN**: This instruction behaves oppositely to **leN**: byte-swapping occurs on little-endian hosts and zero-extension on big-endian hosts.
- **swN**: This instruction always performs a byte-swap, regardless of host endianness.

$$\text{eop}(r_d) \stackrel{\text{def}}{=} \begin{cases} \text{Archi.big_endian} ? \text{byteswapN}(\llbracket r_d \rrbracket_{uN}) : (u64)\llbracket r_d \rrbracket_{uN} & , \text{ if eop} = \text{leN} \\ \text{Archi.big_endian} ? (u64)\llbracket r_d \rrbracket_{uN} : \text{byteswapN}(\llbracket r_d \rrbracket_{uN}) & , \text{ if eop} = \text{beN} \\ \text{byteswapN}(\llbracket r_d \rrbracket_{uN}) & , \text{ if eop} = \text{swN} \end{cases}$$

Formally, we define $\text{byteswapN}(v) = w \equiv \forall i \ k. \ k = N/8 \rightarrow i < k \rightarrow v[i] = w[k - i]$. Here, $v[i]$ denotes the i -th byte of an integer v of width k bytes.

eBPF zero/signed-extension. The eBPF ISA provides zero- and signed-extension operations for fully leveraging modern hardware architectures. We formally define the `signExt` function as follows:

$$\text{signExt}(xop, r_s) \stackrel{\text{def}}{=} \begin{cases} (u32)\llbracket r_s \rrbracket_{sK} & , \text{ if } xop = \text{mov32sxK} \wedge K \in \{8, 16, 32\} \\ \llbracket r_s \rrbracket_{sT} & , \text{ if } xop = \text{mov64sxT} \wedge T \in \{8, 16, 32, 64\} \end{cases}$$

- For the `mov32sxK` instruction, the operation first truncates the 64-bit source register value to a signed K -bit integer, then sign-extends this value to a 32-bit unsigned integer.
- For the `mov64sxT` instruction, the operation only truncates a signed T -bit integer.

3.3.2 Linux eBPF: Memory Semantics. Linux eBPF includes the 128-bit instruction `LDDW` to load a 64-bit integer into a register, which consequently increments the program counter `pc` by 2. Since this 64-bit value can represent either an integer or a pointer, an explicit type-casting operation is required within the standard CompCert memory model. While other research solutions exist for integer-pointer casts, such as [Kim et al. 2025], they are unnecessarily complex for our purposes.

The result of `LDDW` returns a 64-bit integer when the constructor is `const`. Otherwise, the eBPF-specific integer-to-pointer cast, `cast_i2p`, accepts a set of legal memory regions `l_addr` as input, and recursively locates a suitable memory block `h.base_blk` and its corresponding offset. This occurs when v lies within the interval $[h.base_addr, h.base_addr + h.size_blk)$.

$$\begin{aligned} \text{cast_i2p}(v, l_addr) &\stackrel{\text{def}}{=} \text{match } l_addr \text{ with } | [] \Rightarrow \emptyset \\ &| h :: tl \Rightarrow \text{if } h.base_addr \leq v \ \&\& \ v < h.base_addr + h.size_blk \\ &\quad \text{then } [\text{Vptr } h.base_blk \ (v - h.base_addr)] \text{ else } \text{cast_i2p}(v, tl) \end{aligned}$$

The internal instruction `ST_NoSpec` was introduced for eBPF JIT components to mitigate Spectre variant 4 (i.e., Speculative Store Bypass). For the x86 architecture, the eBPF JIT translates it into the `lfence` instruction. For other architectures, the JIT generates no corresponding instructions. Therefore, `ST_NoSpec` is modeled as a no-operation (`nop`) here. Once the eBPF concurrency model is formalized, `ST_NoSpec` could be refined to a low-level semantics that matches its actual behavior.

eBPF Memory Access. As shown in Fig. 6, Linux eBPF provides standard load and store operations with four memory chunks. We formalize these operations using the CompCert memory functions `Mem.loadv` and `Mem.storev`, with minor adaptations to prevent undefined behavior:

- **Load**: After loading a value from CompCert memory, an explicit integer cast is applied. This ensures that the register always holds either a pointer or a 64-bit integer.
- **Store**: When storing a value into memory, CompCert requires casting the value to a 32-bit integer (i.e., $x = u32$) when the memory chunk is not 64-bit.

$$\text{store}(mc, m, addr, v) \stackrel{\text{def}}{=} \text{Mem.storev } mc \ m \ addr \ (\text{match } mc \text{ with } | \text{Mint64} \Rightarrow v \mid _ \Rightarrow (u32) \ v)$$

The Linux eBPF ISA also provides several internal load instructions. `LDSX` behaves like `LD` but performs a signed-extension load for three supported access sizes. `LD_PROBE` and `LDSX_PROBE`

$$\begin{array}{c}
\frac{[[low]]_{u64} \mid_{64} ([[high]]_{u64} \ll_{u64} 32) = v \quad pty = const}{S \xrightarrow{\text{LDDW } pty \ r_d \ low \ high} S[r_d \leftarrow v, pc^{+2}]} \quad \frac{}{S \xrightarrow{\text{ST_NoSpec}} S[pc^{+1}]} \\
\\
\frac{[[low]]_{u64} \mid_{64} ([[high]]_{u64} \ll_{u64} 32) = v \quad pty \neq const \quad \text{cast_i2p}(v, l_addr) = [vp]}{S \xrightarrow{\text{LDDW } pty \ r_d \ low \ high} S[r_d \leftarrow vp, pc^{+2}]} \\
\\
\frac{\text{Mem. loadv}(mc, S.M, [[r_s]] + ofs) = [v]}{S \xrightarrow{\text{LD/LD_PROBE } mc \ r_d \ r_s \ ofs} S[r_d \leftarrow (u64) \ v, pc^{+1}]} \quad \frac{mc \in \{Mint8, Mint16, Mint32, Mint64\} \quad \text{store}(mc, S.M, [[r_d]] + ofs, [[sop]]_x) = [M']}{S \xrightarrow{\text{ST } mc \ r_d \ sop \ ofs} S[pc^{+1}, M \leftarrow M']} \\
\\
\frac{\text{Mem. loadv}(mc, S.M, [[r_s]] + ofs) = [v]}{S \xrightarrow{\text{LDSX/LDSX_PROBE } mc \ r_d \ r_s \ ofs} S[r_d \leftarrow (s64) \ v, pc^{+1}]} \quad \frac{mc \in \{Mint8, Mint16, Mint32, Mint64^*\} \quad \text{Mem. loadv}(mc, S.M, [[r_s]] + ofs) = \emptyset}{S \xrightarrow{\text{LD_PROBE/LDSX_PROBE}^* \ mc \ r_d \ r_s \ ofs} S[r_d \leftarrow 0, pc^{+1}]}
\end{array}$$

Fig. 6. Linux eBPF: MEM semantics

follow the same semantics as normal memory accesses, except that upon failure, they return zero and allow execution to continue rather than halting. Note that the 64-bit memory access behavior of LD_PROBE and LDSX_PROBE is identical; therefore, LDSX_PROBE does not exist for 64-bit accesses.

3.3.3 Semantics: JUMP. Linux eBPF provides branch instructions, shown in Fig. 7, that update only the program counter pc using a computed offset.

- **Unconditional jumps:** The 64-bit instruction JA increments pc by 1 and uses the 16-bit ofs field for short-distance branches. In contrast, the 32-bit instruction JA32 also increments pc by 1 but uses the 32-bit imm field for long-distance branches.
- **Conditional jumps:** These instructions first advance pc to the next instruction. If the condition evaluates to true, they skip $[[ofs]]_{s64}$ additional instructions. The condition evaluation function copN performs a comparison in the signed or unsigned 32/64-bit domain, depending on the specific condition operator cop .

$$\text{copN}(r_d, sop) \stackrel{\text{def}}{=} \begin{cases} [[r_d]]_{uN} <_{uN} [[sop]]_{uN} & , \text{ if } cop \text{ is } lt \\ [[r_d]]_{sN} <_{sN} [[sop]]_{sN} & , \text{ if } cop \text{ is } slt \\ \dots & \\ ([[r_d]]_{sN} \&_N [[sop]]_{sN}) \neq 0 & , \text{ if } cop \text{ is } set \end{cases}$$

In Linux eBPF, function calls and returns are modeled as specialized jump instructions.

eBPF Calls. eBPF provides two external function call instructions: CallID, which invokes a Linux helper function via a static identifier imm , and CallBTF, which calls a helper function via a BPF Type Format identifier imm . We formalize both as CompCert external functions.

The following definitions rely on two auxiliary notations:

- $\text{undef_caller} \stackrel{\text{def}}{=} r_1 \leftarrow \perp, r_2 \leftarrow \perp, r_3 \leftarrow \perp, r_4 \leftarrow \perp, r_5 \leftarrow \perp, r_{11} \leftarrow \perp$ sets the five eBPF caller-saved registers and the hidden register r_{11} to undefined values.
- $\text{undef_callee} \stackrel{\text{def}}{=} r_0 \leftarrow \perp, r_6 \leftarrow \perp, r_7 \leftarrow \perp, r_8 \leftarrow \perp, r_9 \leftarrow \perp, r_{11} \leftarrow \perp$ sets the return value register r_0 , four callee-saved registers, and the hidden register to undefined values.

For internal function calls, also named *BPF-to-BPF* calls, eBPF provides CallInternal. Its behavior follows the standard eBPF calling convention and is formalized by the function `push_frame`:

$$\begin{array}{c}
\frac{[[ofs]]_{s64} + 1 = n}{S \xrightarrow{JA\ ofs} S[pc+n]} \quad \frac{[[imm]]_{s64} + 1 = n}{S \xrightarrow{JA32\ imm} S[pc+n]} \\
\\
\frac{(\text{cop64}(r_d, sop) ? [[ofs]]_{s64} : 0) + 1 = n}{S \xrightarrow{JMP\ cop\ r_d\ sop\ ofs} S[pc+n]} \quad \frac{(\text{cop32}(r_d, sop) ? [[ofs]]_{s64} : 0) + 1 = n}{S \xrightarrow{JMP32\ cop\ r_d\ sopofs} S[pc+n]} \\
\\
\frac{\text{push_frame}(S, imm) = S'}{S \xrightarrow{\text{CallInternal}\ imm} S'} \quad \frac{\text{external_func_by_static_id}(imm, [[r_1]], \dots, [[r_5]]) = \lfloor (v, m) \rfloor}{S \xrightarrow{\text{CallID}\ imm} S[r_0 \leftarrow (u64)\ v, \text{undef_caller}, pc+1, \mathcal{M} \leftarrow m]} \\
\\
\frac{\text{pop_frame}(S) = \lfloor S' \rfloor}{S \xrightarrow{EXIT} S'} \quad \frac{\text{external_func_by_btf_id}(imm, [[r_1]], \dots, [[r_5]]) = \lfloor (v, m) \rfloor}{S \xrightarrow{\text{CallBTF}\ imm} S[r_0 \leftarrow (u64)\ v, \text{undef_caller}, pc+1, \mathcal{M} \leftarrow m]} \\
\\
\frac{\begin{array}{l} [[r_2]] = \text{Vptr } b\ 0 \quad [[r_3]] = \text{Vlong } idx \quad X \in \{32, 64\} \\ \text{Mem.loadv}(\text{MintX}, S, \mathcal{M}, [[r_2]] + idx * (X/8)) = \lfloor npc \rfloor \quad npc = \text{Vptr } b' ofs \end{array}}{S \xrightarrow{\text{TailCall}} S[pc \leftarrow npc, cnt \leftarrow cnt + 1]} \\
\\
\frac{\begin{array}{l} [[r_2]] = \text{Vptr } b\ 0 \quad [[r_3]] = \text{Vlong } idx \quad X \in \{32, 64\} \\ \text{Mem.loadv}(\text{MintX}, S, \mathcal{M}, [[r_2]] + idx * (X/8)) = \lfloor npc \rfloor \quad (npc = \text{Vnullptr} \vee \text{max_call} \leq idx \vee 33 \leq cnt) \end{array}}{S \xrightarrow{\text{TailCall}} S[pc+1]}
\end{array}$$

Fig. 7. Linux eBPF: **JUMP** semantics

- It pushes four caller-saved registers (r_6 through r_9), the current stack pointer r_{10} , and the return address (i.e., $[[pc]] + 1$) onto the top of the host VM call stack $nstk$.
- It allocates a new eBPF local stack of fixed size (512 bytes) for the callee. CompCert's memory model returns a new memory state m' and a block nsp_blk representing this stack.
- It returns a new program state consisting of an updated register map $\mathcal{R}'$, the memory m' , and the new call stack $nstk$. The register map $\mathcal{R}'$ undefines several registers, setting r_{10} to point to the new stack top, and updating pc to the target address (i.e., $[[pc]] + imm$).

eBPF EXIT. For the return instruction EXIT, the transition rule inspects the call stack. If STK is empty, the eBPF program reaches a final state in which pc becomes a null pointer. If the top of STK contains an element h , the current execution corresponds to the end of an internal function call. Following the eBPF calling convention: the return value resides in r_0 , the current local stack is deallocated, the five callee-saved registers (r_1 through r_5) and r_{11} become undefined, the four caller-saved registers (r_6 through r_9) and the previous stack pointer r_{10} are restored from h , and pc

is set to the return address $h.ra$.

```

push_frame( $S, imm$ )  $\stackrel{\text{def}}{=}$ 
  let  $nstk = [ \llbracket r_6 \rrbracket, \llbracket r_7 \rrbracket, \llbracket r_8 \rrbracket, \llbracket r_9 \rrbracket, \llbracket r_{10} \rrbracket, \llbracket pc \rrbracket + 1 ] :: S.STK$  in
  let ( $m', nsp\_blk$ ) = Mem.alloc( $S.M, 0, 512$ ) in
     $S[undef\_callee, r_{10} \leftarrow \forall ptr \ nsp\_blk \ 512, pc \leftarrow \llbracket pc \rrbracket + imm, M \leftarrow m', STK \leftarrow nstk]$ 
pop_frame( $S$ )  $\stackrel{\text{def}}{=}$  match  $S.STK$  with | []  $\Rightarrow S[pc \leftarrow \text{Vnullptr}]$ 
|  $h :: tl \Rightarrow$  match  $\llbracket r_{10} \rrbracket$  with
|  $\forall ptr \ b \_ \Rightarrow$  match Mem.free( $S.M, b, 0, 512$ ) with |  $\emptyset \Rightarrow \emptyset$ 
|  $\llbracket m' \rrbracket \Rightarrow S[undef\_caller, r_6 \leftarrow h.v_6, r_7 \leftarrow h.v_7, r_8 \leftarrow h.v_8, r_9 \leftarrow h.v_9, r_{10} \leftarrow h.v_{10},$ 
 $pc \leftarrow h.ra, M \leftarrow m', STK \leftarrow tl]$ 
|  $\_ \Rightarrow \emptyset$ 

```

eBPF Tail-Call. The eBPF TailCall instruction behaves similarly to a goto statement. It evaluates two pieces of information: register r_2 points to the starting memory block b of the eBPF tail-call jump table `jmp_tbl`, and register r_3 stores the index (idx) of the target eBPF code snippet. If block b contains a valid pointer npc at location idx , the instruction sets the eBPF pc to npc and increments the tail-call counter cnt by one. Otherwise—if npc is a null pointer, the index is out of bounds, or the amount of tail calls exceeds 33^2 —the TailCall instruction sets the eBPF pc to $pc + 1$.

3.4 Formal Specification

One of our goals is to complement the existing textual eBPF standardization drafts and documentation with a precise description using the above semantics. Although prior tools [Mulligan et al. 2014; Owens et al. 2011; Sewell et al. 2007] can automatically generate formal specifications from formal models, we adopt a manual approach to construct a structured table that explicitly links each eBPF instruction to its formal semantics. This table thus serves as a complementary reference.

Table 1 shows representative excerpts. Each row corresponds to one in-kernel eBPF instruction. The first five columns record the opcode and the syntactic constraints on each instruction field. For example, the 64-bit unsigned modulo instruction has opcode `0x9f`. Its destination register range from r_0 to r_9 , excluding r_{10} because r_{10} is read-only. The sixth column defines the formal semantics (for simplification, we use the instruction to represent the corresponding transition rule). The seventh column provides the original eBPF macro in the Linux kernel, for instance, `0x9f` corresponds to `'BPF_ALU64 | BPF_MOD | BPF_X'`.

Table 1. Formal specification (part).

opcode	dst (r_d)	src (r_s)	ofs	imm	semantics (refer to)	Linux eBPF macro
0x9f	[0,9]	[0,10]	0	0	ALU64 $umod \ r_d \ r_s$	BPF_ALU64 BPF_MOD BPF_X
0x97	[0,9]	0	0	$\neq 0$	ALU64 $umod \ r_d \ imm$	BPF_ALU64 BPF_MOD BPF_K
0x97	[0,9]	0	1	$\neq 0$	ALU64 $smod \ r_d \ imm$	BPF_ALU64 BPF_MOD BPF_K

The table-driven specification guides two core tasks:

- *Static Checking in Verifier:* Each instruction must satisfy a well-formed format. *i.e.*, if a binary instruction violates the constraints in the table, the verifier must reject it statically.

²MAX_TAIL_CALL_CNT is 33, see the source <https://elixir.bootlin.com/linux/v7.0-rc3/source/include/linux/bpf.h#L2256>

- *Interpreter Behaviors*: Each well-formed sequential eBPF instruction maps to a precise formal semantics, which ensures deterministic execution behavior.

Synchronization with Linux kernel. As discussed in [Section 1.1](#), frequent kernel updates can introduce inconsistencies between the formal specification (model) and the latest C implementation. To stay synchronized with the Linux upstream, we adopt two steps to monitor each new Linux commit that affects the eBPF test suite, verifier, or interpreter.

- *Benchmarks*: First, we trigger our validation workflow (explained in [Section 4](#)). If the workflow reports that a test fails and the failure does not involve unsupported features (e.g., atomic operations), the issue may stem from an outdated formal model.
- *Source code*: We then manually examine the updated parts (involving the verifier or the interpreter) of each commit for potential inconsistencies.

Whenever a commit changes the behavior of eBPF programs (e.g., by adding new instructions), by observing benchmarks or examining the source code, we must synchronize the corresponding formal model and specification. The current synchronization process requires some manual effort, and a potential solution with higher automation could follow the eBPF continuous integration (CI) workflow [BPF-CI-Maintainers 2025] or build upon some CI-related research e.g., [Bell et al. 2018].

The resulting set of formal specifications for eBPF provides a concise notational reference for eBPF users, maintainers, and verification developers. A long-term goal, requiring further collaboration with the Linux eBPF community, is to establish an eBPF standard with formal specifications analogous to the WASM language [Rossberg 2017]. Nevertheless, the formalization process itself has already contributed to the safety efforts within the Linux eBPF subsystem.

3.5 Formalization Effort for Linux

The semantics model comprises more than 1.5 KLoC of Rocq code, as measured by coqwc (a built-in code statistics tool). During formalization, we identified two previously unknown bugs in the Linux eBPF subsystem. The second bug, deeply hidden, had remained undiscovered for over eight years.

Bug: Reject negative offsets. The previous check ‘`insn->off > 1`’ intended to ensure that the offset was either 0 or 1 for BPF_MOD/BPF_DIV. However, because ‘`insn->off`’ is signed, this condition accepts all negative values (e.g., -1). As a result, malformed instructions may bypass the verifier.

```
- if (insn->imm != 0 || insn->off > 1 ||
+ if (insn->imm != 0 || (insn->off != 0 && insn->off != 1) ||
    (... && opcode != BPF_MOD && opcode != BPF_DIV)) { verbose("BPF_ALU uses reserved fields\n");
```

This issue emerged when maintainers extended the eBPF ISA with signed division and modulo instructions. The revised condition, merged in the Linux upstream, explicitly checks that the offset equals 0 or 1 and rejects all other values. The root cause is the absence of a precise eBPF specification and a unified codebase for instruction validity checks.

Bug: Narrowing conversion in eBPF call offset. The eBPF interpreter’s C implementation stored the `imm` field (a signed 32-bit value containing BPF-to-BPF offset information) in the `off` field (a signed 16-bit value). A problem occurs when the `imm` value falls outside the representable range of `s16`.

```
void bpf_patch_call_args(struct bpf_insn *insn, u32 stack_depth){ ...
- insn->off = (s16) insn->imm;
  insn->imm = xxx - __bpf_call_base_args;
  insn->code = BPF_JMP | BPF_CALL_ARGS; } // opcode = 0xe5, i.e. CallInternal (BPF-to-BPF call)
static u64 __bpf_prog_run(u64 *regs, const struct bpf_insn *insn) { ...
  JMP_CALL_ARGS: // i.e. BPF-to-BPF calls, opcode = BPF_JMP | BPF_CALL_ARGS
    BPF_R0 = (__bpf_call_base_args + insn->imm)(BPF_R1, BPF_R2, BPF_R3, BPF_R4, BPF_R5,
```



```

-               insn + insn->off + 1);
+               insn + AX + 1);

```

This issue has been confirmed by the kernel maintainers. We have proposed a possible solution and are currently discussing it with them. This solution requires two-level fixes: The verifier firstly inserts a ‘MOVI64 AX imm’ instruction before each BPF-to-BPF call to store the imm value into the eBPF hidden register AX (*i.e.*, r_{11}); the interpreter then uses AX for the proper internal branch.

4 Semantics Validation

A formal semantics is useful only if it inspires confidence in its correctness. This requirement becomes critical for a complex system such as Linux eBPF. An ideal approach would prove bisimilarity between our formal model and the actual in-kernel C implementation. However, this approach faces practical challenges, notably the lack of a formal semantics for Linux C (beyond CompCert C). This ambitious, long-term goal lies outside the scope of the present paper.

We therefore validate our eBPF semantics against the original in-kernel C implementation by comparing execution traces. This section presents our validation workflow and its core components.

4.1 Validation Workflow

Key Challenge. Unlike user-space eBPF VMs (*e.g.*, Solana’s implementation, which provides only five simple system calls with deterministic outputs), Linux eBPF exposes more than 200 system calls. Many of these calls, such as `bpf_get_prandom_u32`, return non-deterministic values on each invocation. Consequently, we cannot directly model these calls in Rocq or execute them faithfully in our executable semantics. Moreover, existing validation workflows from prior work on Solana [Yuan et al. 2025] do not readily apply to this setting.

Solution. To address these challenges, we design a specific validation workflow. The key idea is to record a per-step execution trace from the in-kernel eBPF interpreter. We then use these traces to ‘simulate’ the behavior of Linux eBPF system calls within our executable semantics, thereby enabling trace-level equivalence checking.

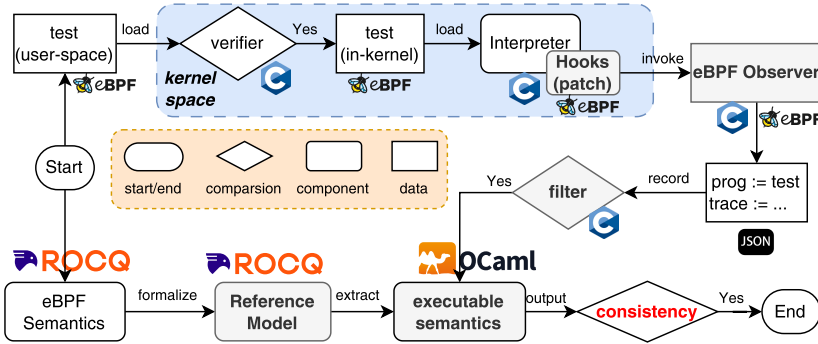


Fig. 8. Semantics Validation Workflow

The validation workflow is illustrated in Fig. 8. Given an eBPF test case from the official Linux eBPF selftest suite [Linux-v7.0 2026a,b], the workflow firstly follows the in-kernel eBPF execution path: If the test case passes the verifier; then, with our specific patch, the generated in-kernel test case loads into the interpreter to produce the per-step execution trace.

eBPF Observer and Hooks. Since the eBPF subsystem provides observation mechanisms within the Linux kernel, our trace recorder—comprising both a kernel-space patch and a user-space

component—leverages eBPF itself to observe the execution of the eBPF VM. The kernel-space patch defines a data structure to record the register state at each step and introduces new eBPF hooks for this purpose. The user-space eBPF program, named `eBPF_observer`, loads into the kernel as a normal eBPF task and executes to collect trace in JSON format whenever these hooks are invoked.

A ‘trick’ aspect is to avoid an infinite call loop involving `eBPF_observer` itself: We introduce an additional flag `bpff_trace_active` to distinguish whether the currently executed eBPF program is `eBPF_observer`. When `bpff_trace_active` is false, the hooks are not invoked.

Filter. Our workflow then applies two filtering rules to tailor the unsupported test case (when filter returns false). Note that the in-kernel verifier may also reject the input test case as the selftest suite includes intentionally invalid eBPF programs.

- *Atomic:* Our current semantics models only sequential eBPF instructions. We therefore omit tests that contain atomic instructions, in particular those with opcodes `0xc3` (32-bit atomic operations) and `0xdb` (64-bit atomic operations).
- *Unknown:* The actual eBPF VM allows to execute some special eBPF test cases which has unknown operations (*i.e.*, undefined behaviors from a formal perspective), detailed in [Section 4.3](#), we also omit those cases.

Trace Information. When the test case pass the filter, the JSON file is loaded into our executable semantics (explained below) for validation. Each JSON file contains several fields: 1) the test program name `prog_name`; 2) the in-kernel eBPF bytecode along with tail-call jump-table bytecode `bpff_prog`; 3) the complete execution trace; and 4) the used memory region information (*e.g.*, the eBPF maps).

Finally, the workflow reports the outcome of each test according to our executable semantics. If it finds a consistent execution, it completes the current workflow and proceeds to the next test. If it detects an inconsistency, it provides feedback to help locate the potential issue.

4.2 Reference Interpreter

Our validation process centers on a reference model extracted from our formal semantics in Rocq. The recursive function `reference_model` executes an eBPF program while checking its behavior against a recorded kernel trace. It takes four parameters: a program state S , the bytecode `prog`, a kernel trace `tr`, and the set of memory regions `l_addr`.

At each step, the reference model uses `find_instr` to decode the current instruction `ins` from `prog` according to the `pc` value. If `ins` is an external call (*i.e.*, `CallID` or `CallBTF`), the reference model inspects the head element `h` of `tr`. When `h` exists, it uses `h` to ‘simulate’ the call: it updates S with the return value recorded in `h`, and moves `pc` to the next instruction. Then recursively invokes itself with the updated state and the remaining trace. For all other instructions, the reference_model executes one semantic step according to the transition rules. It then analyzes the trace `tr`. If `tr` is empty, it checks whether the updated state satisfies `is_final_state`, *i.e.*, the executable counterpart of the predicate `final_state` defined in [Section 3.1](#). Otherwise, it verifies the consistency relation $st \approx_{l_addr} h$ between the updated state `st` and the head element `h` of `tr`.

The relation $st \approx_{l_addr} h$ compares each register value in `st` with the corresponding value in `h`:

- If r_i is a 64-bit integer, the relation directly compares r_i with $h.reg[i]$.
- If r_i is a pointer, the relation first casts the 64-bit trace value to a pointer using `l_addr` and then performs the comparison.
- If r_i is undefined (*e.g.*, caller-saved registers after a function call), the relation returns true.
- Any other case indicates a potential inconsistency.

The expected Linux eBPF verifier should reject programs that use undefined values. *i.e.*, we assume the current in-kernel verifier enforces this property. For simplicity, we omit the construction

of memory regions and the initial program state. We also omit auxiliary implementation details, e.g., JSON parsing, from this presentation.

```

reference_model( $S, prog, tr, l\_addr$ )  $\stackrel{\text{def}}{=}$ 
  match find_instr( $S, prog$ ) with |  $\emptyset \Rightarrow false$ 
  | [ $ins$ ]  $\Rightarrow$  match  $ins$  with
    | CallID _ | CallBTF _  $\Rightarrow$  match  $tr$  with | []  $\Rightarrow false$ 
    |  $h :: tl \Rightarrow$  let  $st = \mathcal{S}[undef\_caller, r_0 \leftarrow h.reg[0], pc \leftarrow \llbracket pc \rrbracket + 1]$  in
      reference_model( $st, prog, tl, l\_addr$ )
    | _  $\Rightarrow$  let  $st = \text{exec\_step}(S, ins)$  in match  $tr$  with
      | []  $\Rightarrow$  is_final_state( $st$ )
      |  $h :: tl \Rightarrow$  if  $st \approx_{l\_addr} h$  then reference_model( $st, prog, tl, l\_addr$ ) else false

 $S \approx_{l\_addr} h \stackrel{\text{def}}{=} \forall r_i. \text{match } \llbracket r_i \rrbracket \text{ with}$ 
  | Vlong  $v \Rightarrow v = h.reg[i]$ 
  | Vptr  $b$  ofs  $\Rightarrow \llbracket r_i \rrbracket = \text{cast\_i2p}(h.reg[i], \llbracket pc \rrbracket, l\_addr)$ 
  |  $\perp \Rightarrow true$  | _  $\Rightarrow false$ 

validation_workflow( $json\_file$ )  $\stackrel{\text{def}}{=}$ 
  let ( $l\_addr, st$ ) = build_region_and_init_state( $json\_file$ ) in
    reference_model( $st, json\_file.bpf\_prog, json\_file.trace, l\_addr$ )

```

Limitations. We clarify the limitations of the proposed validation workflow.

- *Benchmarks:* We only select two official Linux eBPF test suites [Linux-v7.0 2026a,b] for validation in this paper. However, we expect no difficulty in extending our approach to other Linux eBPF benchmark suites, because the workflow captures all traces from kernel space.
- *Equivalence Relation:* We compare only the per-step register state and omit the memory state from this comparison. Recording and comparing the entire memory regions that eBPF uses would be time-consuming. Therefore, we rely on register-level comparison to indirectly check the status of specific memory regions. Our workflow assumes that external calls have no side effects. Consequently, when an eBPF external call modifies memory, one should complement the validation process with the relevant memory changes.

4.3 Validation Results

Our validation workflow implementation mainly includes: i) the kernel patch, comprising 200+ lines of C code; ii) the eBPF Observer, comprising 600+ lines of C code; iii) the OCaml auxiliary functions, comprising 150+ lines of code. The validation process executes 1700+ tests within the extracted OCaml executable semantics. These include 1100+ tests drawn from the Linux eBPF test suite [Linux-v7.0 2026b] and 600+ tests specifically for the eBPF interpreter [Linux-v7.0 2026a].

The validation workflow uncovers subtle inconsistencies in our initial semantics model and guides subsequent refinements. We highlight representative examples below.

Inconsistencies Found: the hidden eBPF register. The workflow reveals that our initial model omits the hidden eBPF register AX (i.e., r_{11}). In several benchmarks, e.g., those involving division, the verifier inserts additional instructions that use AX to maintain semantic alignment. Because our initial model lacks this register, the validation process exposes the mismatch.

Inconsistencies Found: Undefined Behaviors. During validation, we observed that eBPF programs can run in either unprivileged or privileged mode. Privileged mode may permit undefined behaviors. The following code, taken from the official selftest suite, illustrates this issue. The program accesses an uninitialized memory region and retrieves an unknown value, whose behavior is undefined from a formal perspective (e.g., the standard CompCert memory model). It then performs 64-bit ALU operations on this value, which also produce undefined results in CompCert.

```
__success __retval(0) // On privileged mode: success, the expected return value is 0;
__failure_unpriv __msg_unpriv("... access prohibited for !root") // On unprivileged mode: failure
__naked void stack_read_priv_vs_unpriv(void) { asm volatile (" ...
    r2 = *(u32*)(r1 + 0); /* Get an unknown value. */ // i.e. 'LD Mint32 r2 r1 0'
    r2 &= 4; /* Make it small and 4-byte aligned. */ // i.e. 'ALU64 and r2 4'
    r2 += r10; // i.e. 'ALU64 add r2 r10'
    r0 = *(u32*)(r2 + 0); /* dereference it for a stack read */ // i.e. 'LD Mint32 r0 r2 0'
    r0 = 0; // i.e. 'MOVI64 r0 0'
    exit; ... // i.e. 'EXIT'
```

These observations help define the scope of this paper. Our formal model focuses on well-defined eBPF interpreter behaviors from a formal perspective. Therefore, the validation excludes tests with unknown behaviors. We argue that such behaviors require careful manual inspection by experts.

Inconsistencies Found: Linux eBPF Implementation. During validation, we also discovered inconsistencies between the Linux eBPF interpreter and its target JIT. While those may be familiar to eBPF kernel maintainers (or have designed them intentionally), it remains unclear to normal eBPF users and verification engineers. For instance, the Linux eBPF model allows mixing tail-calls with BPF-to-BPF calls, but the interpreter does not support this combination, and the in-kernel eBPF verifier rejects such programs [Linux-verifier 2026].

Remark. The faithfulness of our semantics is guaranteed at two levels: i) thorough testing via the official Linux selftests reduces hidden inconsistencies; ii) the application presented in the next section further builds confidence in the accuracy of our formalization.

5 Application

This section demonstrates the practical utility of our semantics.

5.1 Mechanized Soundness Proofs of the tnum Abstract Domain

The tnum (tristate numbers) abstraction in the Linux eBPF verifier tracks each bit of an n -bit integer variable. Each bit is either known (0 or 1) or unknown, denoted by μ . The verifier represents an n -bit tnum value t using two n -bit fields: value and mask. Let $t[i]$ denote the i th bit of t . Formally,

$$\forall i \in [0, n-1]. t[i] \stackrel{\text{def}}{=} \begin{cases} 0, & \text{if } t.\text{value}[i] = 0 \wedge t.\text{mask}[i] = 0, \\ 1, & \text{if } t.\text{value}[i] = 1 \wedge t.\text{mask}[i] = 0, \\ \mu, & \text{if } t.\text{value}[i] = 0 \wedge t.\text{mask}[i] = 1. \end{cases}$$

Intuitively, value records known bits, while mask marks unknown bits. For example, the tnum value $t = 0b1\mu0\mu$ with value = 0b1000 and mask = 0b0101 represents the concrete set {8, 9, 12, 13}. A tnum is not well-formed, i.e., it denotes the bottom element, iff $t.\text{value} \ \& \ t.\text{mask} \neq 0$.

Mechanized Soundness Proof. We formalize in Rocq all tnum operations implemented in the eBPF verifier, along with the swb domain—a recently generalized tnum theory [Fan et al. 2025].

- Unary eBPF operations: tnum_neg, tnum_bswap16, tnum_bswap32 and tnum_bswap64;
- Binary operations tnum_op where
 - eBPF: op $\in \{\text{lshift}, \text{rshift}, \text{arshift}, \text{add}, \text{sub}, \text{and}, \text{or}, \text{xor}, \text{mul}\}$;

– swb: tnum_udiv and tnum_umod.

Definition 5.1 (Membership). $\text{mem_tnum}(i, a)$ holds iff the concrete value i belongs to the set represented by the abstract tnum a . Formally, $\text{mem_tnum}(i, a) \stackrel{\text{def}}{=} (i \& \sim a.\text{mask}) == a.\text{value}$.

We prove the soundness of each tnum operation in Rocq, *i.e.*, abstract execution over-approximates (*i.e.*, the *Membership* relation) concrete execution.

THEOREM 5.2 (SOUNDNESS OF tnum_op). *Let tnum_op be an abstract binary operator and op its concrete counterpart. If the inputs satisfy Membership, then the output also satisfies Membership.*

$$\forall a \ b \ i \ j. \text{mem_tnum}(i, a) \wedge \text{mem_tnum}(j, b) \implies \text{mem_tnum}((\text{op } i \ j), (\text{tnum_op } a \ b)).$$

Note that the soundness theorem for unary tnum_op follows a similar structure. For the eBPF binary tnum_op, the concrete operations directly adhere to the Linux eBPF ALU semantics defined in Fig. 5. In contrast, for the swb domain, the concrete operations adopt the general semantics of division and modulo rather than the eBPF-specific ones.

Discussion. Our proof differs from prior tnum research (AGNI [Vishwanathan et al. 2022, 2023] and swb) in two aspects:

- *Complete operations:* Prior work verifies only a subset of tnum operations, excluding three byte-swap and two division/modulo functions. The original swb paper also provides no soundness proofs for all tnum operations. In contrast, we verify all of them.
- *Verification capability:* We use Rocq to leverage its full verification power, proving the soundness of two tnum division and modulo operations of swb. Prior work relies on the Z3 SMT solver [de Moura and Bjørner 2008] and could easily extend support to the three tnum byte-swap functions. However, we observe that proving the actual tnum division and modulo operations is difficult for SMT solvers using built-in bitvector theories. Our experiment shows that with a bit-size of 8, Z3 can solve the division and modulo soundness. Yet when the bit-size increases to 64, we observe no result within 24 hours on a high-performance server (AMD Processor with 256 CPUs, max 4409 MHz, and 1.5TB memory).

To clarify, our verification work complements the prior effort of AGNI: it not only re-proves its SMT-based verification results in Rocq but also provides a verification path that enhances the verification capability when the underlying SMT solvers reach their limits.

5.2 Verification Effort for Tnum Theory

Our Rocq implementation of tnum soundness comprises 1432 lines of specification and 3994 lines of proof. During formalization, we identified several issues in the eBPF verifier and in the swb domain.

Linux eBPF Precise: add new abstract operations. We found that the Linux range analysis does not properly support eBPF byte-swap instructions that are common in networking code. Consequently, the verifier loses precision when programs contain these operations. To address this limitation, we design and implement new abstract operations. The Linux upstream has merged our improvements: *i.e.*, extending the tnum domain with precise range analysis for all three byte-swap instructions.

We take tnum_bswap16 from the patch code for explanation. It first aligns the value and mask fields of tnum with 16 bits, and then invokes the Linux system call swab16 on both tnum fields.

```
+ struct tnum tnum_bswap16(struct tnum a) {
+   return TNUM(swab16(a.value & 0xFFFF), swab16(a.mask & 0xFFFF));
```

In addition, we have also introduced other eBPF-specific integer interval algorithms (word-level, not tnum bit-level) into the Linux upstream to handle cases such as division. We omit the details of these algorithms for simplicity.

We further formalized and evaluated the generalized tnum theory swb in Rocq. Our investigation uncovered soundness and optimization issues in its code. The authors have confirmed these findings.

Soundness issues. swb provides analysis algorithms for general tnum division and modulo operations. For instance, the function `remGetLowBits(LSH, RSH)` computes the known low bits of the tnum modulo result. The main idea is that in the modulo operation `LHS % RHS`, when `RHS` is an even number with `qzero` consecutive low zero bits, the result depends only on the low `qzero` bits of `LHS`. However, if uncertainty originally exists in the high bits of `LHS` (where mask bits are 1), this uncertainty should propagate to the entire tnum result. The original implementation suffers from a soundness issue (line 8) because it fails to apply `(~ mask)`, which sets all high bits to 1.

```

1 tnum<Number>::remGetLowBits(const tnum<Number> &LHS, const tnum<Number> &RHS) const {
2   if(!RHS.is_zero() && RHS.value.is_even() && RHS.mask.is_even()) { ...
3   -   if(qzero == 0) return top(w);
4   -   wrapint mask(((uint64_t)1 << (uint64_t)(qzero - 1)) - 1), w);
5   +   wrapint mask(((uint64_t)1 << (uint64_t) qzero) - 1), w);
6     wrapint res_value = LHS.value & mask;
7     wrapint res_mask = LHS.mask & mask;
8   -   tnum<Number> res(res_value, res_mask);
9   +   tnum<Number> res(res_value, res_mask | (~ mask));
10    return res;

```

Optimization issues. Our evaluation also reveals optimization gaps in the swb implementation.

- *Dead code:* For example, `remGetLowBits` includes a conditional statement (line 3) to check `'qzero == 0'`. However, according to line 2, we know `RSH` is non-zero and even, making this condition always false.
- *Imprecise design:* For instance, an unnecessary `'qzero - 1'` (line 5) causes the result to always lose one known bit.

6 Discussion: Linux eBPF Concurrency Model

The eBPF ISA provides support for atomic memory accesses (both full and relaxed ordering) but also acquire/release semantics that are implemented using various fences depending on the architecture. Despite recent progress, the expected concurrent eBPF semantics remains an open question [McKenney and Mohan 2024]. The eBPF memory model must satisfy several key requirements:

LKMM compatibility The eBPF memory model should align with the existing Linux Kernel Memory Model (LKMM) [Alglave et al. 2018];

ISA friendly the eBPF memory model should be easily implementable for various ISA, notably x86, ARM, Power and RISC-V;

Verifier friendly the eBPF in-kernel verifier should be sound without any knowledge of concurrent eBPF programs.

Because eBPF programs run inside the kernel, the LKMM could be an obvious choice. However, according to the LKMM, some racy programs do not have a defined semantics. For the kernel, this is not an issue because it is the role of kernel developers to make sure that their code adheres to the LKMM. For eBPF, this is not satisfactory because the in-kernel verifier would need to reject such undefined racy programs. The current verifier cannot perform this task, as it analyzes only one eBPF program at a time.

Another approach would adopt a highly restrictive memory model and assume Sequential Consistency (SC). In this simple model, the semantics is given by the interleaving of the concurrent eBPF programs. the drawback is that enforcing this idealized model on contemporary architectures which provides much weaker memory models would require the JIT to protect concurrent memory

accesses using costly synchronization primitives. Our Rocq implementation do formalize such a SC semantics for eBPF programs, but we omit its presentation due to the limitations discussed above.

A more promising solution might equip eBPF with a weak memory model at the ISA level. The ARM memory model [Armstrong et al. 2019; Pérami et al. 2026; Pulte et al. 2017; Simner et al. 2022] presents a viable candidate, as its relative weakness makes it easier for stronger models like Total Store Ordering (TSO) [Ševčík et al. 2013] to implement. Though it looks feasible to provide an operational semantics, maybe using an explicit reorder buffer [Colvin and Su 2025], it looks like an open challenge to effectively run such semantics on realistic programs, beyond litmus tests.

7 Related Work

We examine research on the formal verification of Linux eBPF and its ISA variants. We focus primarily on the following aspects relevant to our main contributions: i) *mechanized semantics*: the instruction scope and form (implementation language) of the semantics used; ii) *consistency*: whether the work employs semantics validation or formal equivalence proofs; iii) *application*: the real-world applications enabled by the semantics.

7.1 Verifier-Enhanced Verification

Multiple approaches aim to enhance the existing verifier. Some employ fuzzing [Lyu et al. 2025; Sun et al. 2024] or state embedding [Sun and Su 2024]. Others adopt formal verification to propose new workflows that replace or augment the current one.

PREVAIL [Gershuni et al. 2019] presents an eBPF verifier that uses the zone abstract domain for static analysis. The eBPF-in-Windows project [Microsoft 2019] has adopted this verifier. A recent work, VEREFINE [Hamza et al. 2025], builds on PREVAIL’s parsing backend to design a refinement type system for eBPF programs. VEREFINE defines type rules for ALU, JUMP, and memory instructions, ensuring memory safety and information flow safety. Both tools are implemented in C++.

AGNI [Vishwanathan et al. 2023] offers another verification approach for eBPF. It uses the Z3 SMT solver to prove the soundness of range domains—specifically, word-level integer intervals and bit-level tnum [Vishwanathan et al. 2022]—in the Linux eBPF verifier. At the time of writing, AGNI supports 16 ALU and 18 JUMP eBPF instructions for Linux version v6.12, encoding the corresponding semantics in SMT2 format.

EXOVERIFY [Luke et al. 2021] introduces a proof-carrying code (PCC) framework for Linux eBPF programs. It encodes the semantics of 85 eBPF instructions in Lean to generate proofs. Similarly, VEP [Wu et al. 2025] written in C, is another PCC framework that translates annotated C code to eBPF bytecode with accompanying proofs. The Linux kernel then re-checks these proofs. BCF [Sun and Su 2025], implemented in C, proposes a proof-guided abstraction refinement technique. This approach enhances the Linux eBPF verifier with additional proofs produced in user space.

Several other formal verification projects also relate to the eBPF verifier. For instance, BPFVERIFY [Bromberger et al. 2024] presents a translation from a subset of eBPF instructions to constrained horn clause sets. Sanjit and Hovav [Sanjit and Hovav 2023] propose a range analysis approach, similar to AGNI, that translates a set of eBPF arithmetic instructions into SMT format.

In short, most formal verification efforts for the Linux eBPF verifier do not aim for completeness of the mechanized eBPF semantics. Few works, such as PREVAIL, support eBPF atomic opcodes in their source implementation. However, it remains unclear how they address the eBPF concurrency model. The closest work in verifier-related verification, EXOVERIFY, considers only a subset of the user-space eBPF ISA without consistency guarantees. In contrast, our Rocq-mechanized model covers all in-kernel non-atomic eBPF instructions, amounting to 160 of the 172 instructions. A subsequent validation process further increases confidence in the accuracy of our model. For the uncovered atomic parts, we discuss the eBPF concurrency model and outline our potential solution.

7.2 eBPF-related Compiler Verification

JITTERBUG [Nelson et al. 2020] presents a framework for generating automated proofs for real-world Linux eBPF JIT compilers. The work implements a 32-bit RISC-V eBPF JIT that has been upstreamed to the Linux kernel. JITTERBUG extends SERVAL [Nelson et al. 2019], a symbolic evaluation tool with semantics for eBPF ALU instructions, to support a large subset of the Linux eBPF ISA—specifically, 117 instructions—in Rosette [Torlak and Bodik 2013], a solver-aided programming language.

JITSYNTH [Van Geffen et al. 2020] introduces a new technique for synthesizing verified JIT compilers, and applies it to synthesize a RISC-V compiler from Linux eBPF. JITSYNTH provides a mechanized eBPF semantics in Lean, covering 87 eBPF instructions.

K2 [Xu et al. 2021] is a program-synthesis-based compiler that targets eBPF optimization with formal guarantees. It currently covers a small subset of the eBPF ISA without mechanization.

Several other verification efforts target eBPF ISA families.

JITK [Wang et al. 2014] is the first verified BPF JIT compiler that translates classical BPF (not eBPF) bytecode into CompCert’s intermediate language. JITK formalizes the semantics of BPF in Rocq, covering 2 registers and 43 instructions.

RIOT-OS eBPF [Zandberg et al. 2022] supports a core subset of the Linux eBPF ISA, consisting of 88 user-space instructions. Although the CertrBPF project [Yuan et al. 2022] formalizes the full semantics of RIOT-OS eBPF in Rocq, its behavior does not align with Linux eBPF. One key reason for this discrepancy is that RIOT-OS eBPF employs dynamic defensive memory semantics at runtime, whereas Linux eBPF avoids such complex semantics because its verifier statically rejects any potentially unsafe memory access. RIOT-OS eBPF also includes a verified JIT compiler [Yuan et al. 2024] in Rocq that translates ALU instructions to ARM targets.

Solana eBPF [Solana-labs 2018] originally derived from user-space Linux eBPF. However, its instruction behavior and instruction types diverge significantly to meet the requirements of its blockchain application. Solana eBPF has a complete mechanized semantics (named Solana-in-Isabelle/HOL with 112 opcodes) [Yuan et al. 2025] and a verified per-JIT compiler in Isabelle/HOL, but it lacks a formal document for development.

CompCert-BPF [Priya et al. 2025] is a verified compiler that extends the standard CompCert framework with a new eBPF-like backend. It formalizes a subset of eBPF instructions in Rocq, associating them with CompCert-specific pseudo-instructions. However, its ISA semantics adapts to the CompCert framework and does not faithfully reflect the behavior of eBPF bytecode executed in the Linux kernel. For instance, in CompCert-BPF, a call instruction (which does not distinguish between BPF-to-BPF calls and system calls, and lacks tail calls) modifies only the eBPF *pc* and an auxiliary return address register *ra* (which does not exist in actual in-kernel eBPF). Additionally, the eBPF stack frame allocation is performed by another pseudo-instruction called *Palloccframe*. Furthermore, a division instruction encountering division-by-zero gets stuck. In contrast, actual Linux eBPF division-by-zero returns zero, which aligns with the semantics we define in this paper.

Except for textual documents—e.g., the eBPF standard draft [Thaler 2024] and the de facto standard in the Linux kernel [Linux-Maintainers 2026]—the most closely related eBPF research are CertrBPF, Solana-in-Isabelle/HOL, and CompCert-BPF. We compare them to our formal semantics on the following three directions that reflect the primary contributions in this paper:

- *Mechanized Semantics*: The three mechanized semantics focus on user-space eBPF ISA variants. One cannot directly extend these existing works to produce the Linux eBPF in-kernel semantics for two reasons. First, user-space eBPF and in-kernel eBPF are different (e.g., the hidden register r_{11}). Second, these eBPF variants have diverged from Linux eBPF. For example, Solana eBPF performs 32-bit signed addition, whereas Linux eBPF performs 32-bit unsigned

addition for the same opcode. In contrast, our model focuses on the mechanized in-kernel eBPF semantics and covers all 160 sequential instructions of Linux eBPF.

- *Consistency*: CertrBPF presents an end-to-end verification workflow to produce a verified C implementation from its Rocq model, but it does not thoroughly discuss the consistency between the generated C code and the original one. CompCert-BPF, in contrast, is not consistent with the actual in-kernel eBPF. Solana-in-Isabelle/HOL and our model both trust the extraction mechanism of proof assistants and validate consistency through extensive testing. A key difference from the Solana semantics validation lies in the complexity and randomness inherent in the Linux kernel. These characteristics demand a fundamentally different validation approach to address the associated challenges. Furthermore, our validation workflow achieves greater precision than that of Solana-in-Isabelle/HOL: it compares only the final output, whereas we compare the internal state at each step.
- *Usability*: CertrBPF and Solana-in-Isabelle/HOL apply their semantics to prove the safety property of verifier components (both verifiers only perform basic instruction format checking) as well as JIT (ALU-part) correctness. CompCert-BPF is the first verified eBPF compiler from C code. In comparison, our semantics are used to prove the soundness of key abstract operations used in the actual Linux verifier, to address the real-world vulnerabilities, and to improve the precision of Linux and the generalized tnum theory swb.

Overall, the key differentiator of our effort compared to the existing work, as cited above, is that our semantics achieves: (A) mechanized semantics that models in-kernel eBPF instructions; (B) consistency that precisely reflects the actual Linux eBPF behaviors; (C) applicability to formal reasoning analysis on real-world eBPF in-kernel verifier.

8 Conclusion and Future Work

This paper presents an operational, mechanized semantics for the Linux eBPF ISA—the first step towards formal verification of the in-kernel verifier. Our semantics covers all 160 sequential instructions and provides their formal specification. To guarantee faithfulness, we thoroughly validate the semantics through a novel consistency-checking workflow and apply our formal model to support real-world verification efforts in the Linux community.

Our future work focuses on the following research directions.

eBPF Concurrency Model. We are extending our work toward a complete concurrency model for eBPF based on the semantics developed in this paper. This effort represents the final step toward a complete formalization of Linux eBPF semantics. It is expected to provide a solid foundation to pursue two complementary objectives: 1) mechanizing the axiomatic semantics of LKMM in Rocq; and 2) extending a small-step, thread-local eBPF semantics that follows a style close to LKMM and over-approximates the axiomatic model. Establishing the correspondence between these two semantics requires a substantial proof effort. Nevertheless, this design offers two advantages. First, the Linux community already accepts the LKMM axiomatic semantics. Second, the verification community can reason more conveniently about the verifier and JITs with a small-step model.

Fully Verified eBPF Verifier. Our long-term goal is to prove the correctness of the entire in-kernel eBPF verifier in Rocq. The semantics presented in this paper provides the necessary foundation for this objective. To move forward, we plan to develop a proof-carrying code framework that integrates a proof-checking mechanism into the Linux kernel. As an initial step, we are formalizing the type systems implemented by the current eBPF verifier. These systems include the bit-level tnum abstraction (described in [Section 5.1](#)), the word-level abstraction (integer interval domains), and the pointer-type discipline. In addition, a data flow analysis for stack slot liveness [[Linux 2025](#)]

was recently introduced into the in-kernel verifier. We plan to prove its correctness in Rocq, as another step toward enhancing the safety of the in-kernel verifier.

Fully Verified eBPF Compilers. Extending the CompCert-BPF project to align with the actual Linux eBPF semantics proposed in this paper would provide a verified eBPF compiler—an interesting direction for future work. Additionally, the in-kernel eBPF verifier itself functions as a compiler. Proving its compilation correctness through translation validation [Pnueli et al. 1998], treating the verifier as a black box, represents another meaningful research avenue. We intend to draw upon CompCert’s validation approach [Rideau and Leroy 2010] to complete such a proof.

Acknowledgments

This work has been partially funded by The authors would like to thank ...

Data-Availability Statement

The source Rocq code, proofs, generated code, and benchmark data are available as an anonymized repository [Anonymised 2026]. A self-contained artifact (e.g., including a VM image, the complete formal specification, and so on) will be submitted for the Artifact Evaluation.

References

- Jade Alglave, Luc Maranget, Paul E. McKenney, Andrea Parri, and Alan Stern. 2018. Frightening Small Children and Disconcerting Grown-ups: Concurrency in the Linux Kernel. *SIGPLAN Not.* 53, 2 (March 2018), 405–418. <https://doi.org/10.1145/3296957.3177156>
- Anonymised. 2026. A Mechanized Operational Semantics of Linux eBPF ISA and its Applications. <https://anonymous.4open.science/r/linux-ebpf-semantics-D430/>
- Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA semantics for ARMv8-a, RISC-v, and CHERI-MIPS. *Proc. ACM Program. Lang.* 3, POPL, Article 71 (Jan. 2019), 31 pages. <https://doi.org/10.1145/3290384>
- Jonathan Bell, Owolabi Legunsen, Michael Hilton, Lamyaa Eloussi, Tiffany Yung, and Darko Marinov. 2018. DeFlaker: automatically detecting flaky tests. In *Proceedings of the 40th International Conference on Software Engineering (Gothenburg, Sweden) (ICSE '18)*. Association for Computing Machinery, New York, NY, USA, 433–444. <https://doi.org/10.1145/3180155.3180164>
- Yves Bertot and Pierre Castran. 2010. *Interactive Theorem Proving and Program Development: Coq'Art The Calculus of Inductive Constructions* (1st ed.). Springer Publishing Company, Incorporated, Berlin, Heidelberg.
- BPF-CI-Maintainers. 2025. BPF CI GitHub Actions workflows. https://github.com/kernel-patches/bpf/tree/bpf-next_base Accessed: 2026-03-14.
- Martin Bromberger, Simon Schwarz, and Christoph Weidenbach. 2024. Automatic Bit- and Memory-Precise Verification of eBPF Code. In *Proceedings of 25th Conference on Logic for Programming, Artificial Intelligence and Reasoning*, Vol. 100. EPiC Series in Computing, Port Louis, Mauritius, 198–221. <https://doi.org/10.29007/sj4l>
- Robert J. Colvin and Roger C. Su. 2025. Structural Operational Semantics for Functional and Security Verification of Pipelined Processors. In *Computer Aided Verification*, Ruzica Piskac and Zvonimir Rakamarić (Eds.). Springer Nature Switzerland, Cham, 363–388.
- Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Los Angeles, California) (POPL '77). Association for Computing Machinery, New York, NY, USA, 238–252. <https://doi.org/10.1145/512950.512973>
- Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 337–340.
- Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. The Lean Theorem Prover (System Description). In *Automated Deduction - CADE-25*, Amy P. Felty and Aart Middeldorp (Eds.). Springer International Publishing, Cham, 378–388.
- Srikar Dronamraju. 2012. Uprobe-tracer: Uprobe-based Event Tracing. <https://docs.kernel.org/trace/uprobetracer.html> Accessed: 2026-03-14.

- Guangsheng Fan, Liqian Chen, Banghu Yin, Wenyu Zhang, Peisen Yao, and Ji Wang. 2025. Program Analysis Combining Generalized Bit-Level and Word-Level Abstractions. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA030 (June 2025), 23 pages. <https://doi.org/10.1145/3728905>
- Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzy, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (PLDI 2019). Association for Computing Machinery, New York, NY, USA, 1069–1084. <https://doi.org/10.1145/3314221.3314590>
- Google-Inc. 2023. CVE-2023-2163. <https://nvd.nist.gov/vuln/detail/CVE-2023-2163> Accessed: 2026-03-14.
- Sudhanshu Goswami. 2005. An introduction to KProbes. <https://lwn.net/Articles/132196/> Accessed: 2026-03-14.
- Ameer Hamza, Lucas Zavalía, Arie Gurfinkel, Jorge A. Navas, and Grigory Fedyukovich. 2025. A Flow-Sensitive Refinement Type System for Verifying eBPF Programs. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 410 (Oct. 2025), 28 pages. <https://doi.org/10.1145/3763799>
- Kaiming Huang, Mathias Payer, Zhiyun Qian, Jack Sampson, Gang Tan, and Trent Jaeger. 2025. SoK: Challenges and Paths Toward Memory Safety for eBPF. In *IEEE Symposium on Security and Privacy, SP 2025, May 12-15, 2025*, Marina Blanton, William Enck, and Cristina Nita-Rotaru (Eds.). IEEE, San Francisco, CA, USA, 848–866. <https://doi.org/10.1109/SP61157.2025.00134>
- Yonghyun Kim, Minki Cho, Jaehyung Lee, Jinwoo Kim, Taeyoung Yoon, Youngju Song, and Chung-Kil Hur. 2025. Archmage and CompCertCast: End-to-End Verification Supporting Integer-Pointer Casting. *Proc. ACM Program. Lang.* 9, POPL, Article 45 (Jan. 2025), 29 pages. <https://doi.org/10.1145/3704881>
- James C King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (1976), 385–394.
- Xavier Leroy, Andrew W. Appel, Sandrine Blazy, and Gordon Stewart. 2012. *The CompCert Memory Model, Version 2*. Research Report RR-7987. INRIA. 26 pages. <https://inria.hal.science/hal-00703441>
- Xavier Leroy and Sandrine Blazy. 2008. Formal verification of a C-like memory model and its uses for verifying program transformations. *JAR* 41, 1 (2008), 1–31.
- Linux. 2025. Linux eBPF stack liveness analysis. <https://lore.kernel.org/all/20250918-callchain-sensitive-liveness-v3-6-c3cd27bacc60@gmail.com/> Accessed: 2026-03-14.
- Linux-Maintainers. 2026. BPF Instruction Set Architecture (ISA) draft-ietf-bpf-isa-04. <https://github.com/torvalds/linux/blob/1c9982b4961334c1edb0745a0cabd34bc2de675/Documentation/bpf/standardization/instruction-set.rst> Accessed: 2026-03-14.
- Linux-v7.0. 2026a. Linux eBPF interpreter suite. https://github.com/torvalds/linux/blob/master/lib/test_bpf.c Accessed: 2026-03-14.
- Linux-v7.0. 2026b. Linux eBPF selftest suite. <https://github.com/torvalds/linux/tree/master/tools/testing/selftests/bpf> Accessed: 2026-03-14.
- Linux-verifier. 2026. Linux eBPF Implementation: Verifier rejects the mix tail-calls with BPF-to-BPF calls. <https://elixir.bootlin.com/linux/v7.0-rc3/source/kernel/bpf/verifier.c#L10293> Accessed: 2026-03-14.
- Nelson Luke, Wang Xi, and Torlak Emina. 2021. A proof-carrying approach to building correct and flexible BPF verifiers. <https://github.com/uw-unsat/exoverifier/> Accessed: 2026-03-14.
- Tao Lyu, Kumar Kartikeya Dwivedi, Thomas Bourgeat, Mathias Payer, Meng Xu, and Sanidhya Kashyap. 2025. eBPF Misbehavior Detection: Fuzzing with a Specification-Based Oracle. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) (SOSP '25). Association for Computing Machinery, New York, NY, USA, 701–718. <https://doi.org/10.1145/3731569.3764797>
- Steven McCanne and Van Jacobson. 1993. The BSD Packet Filter: A New Architecture for User-level Packet Capture. In *Usenix Winter Conference*, Vol. 46. USENIX, San Diego, California, USA, 259–270.
- Paul E. McKenney and Puranjay Mohan. 2024. Instruction-level BPF memory model. https://bpfconf.ebpf.io/bpfconf2024/bpfconf2024_material/bpfmemmodel.2024.05.13c.pdf Linux Kernel Developers' bpfconf2024. Accessed: 2026-03-14.
- Microsoft. 2019. eBPF implementation that runs on top of Windows. <https://github.com/microsoft/ebpf-for-windows> Accessed: 2026-03-14.
- MITRE-Corporation. 2016. CVE-2016-4557. <https://www.cve.org/CVERecord?id=CVE-2016-4557> Accessed: 2026-03-14.
- MITRE-Corporation. 2021. CVE-2021-3490. <https://www.cve.org/CVERecord?id=CVE-2021-3490> Accessed: 2026-03-14.
- Mohamed Husain Noor Mohamed, Xiaoguang Wang, and Binoy Ravindran. 2023. Understanding the Security of Linux eBPF Subsystem. In *APSys '23: Proceedings of the 14th ACM SIGOPS Asia-Pacific Workshop on Systems* (Seoul, Republic of Korea). Association for Computing Machinery, New York, NY, USA, 87–92. <https://doi.org/10.1145/3609510.3609822>
- Dominic P. Mulligan, Scott Owens, Kathryn E. Gray, Tom Ridge, and Peter Sewell. 2014. Lem: reusable engineering of real-world semantics. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming* (ICFP '14). Association for Computing Machinery, New York, NY, USA, 175–188. <https://doi.org/10.1145/2628136.2628143>
- Luke Nelson, James Bornholt, Ronghui Gu, Andrew Baumann, Emina Torlak, and Xi Wang. 2019. Scaling Symbolic Evaluation for Automated Verification of Systems Code with Serval. In *Proceedings of the 27th ACM Symposium on Operating Systems*

- Principles* (Huntsville, Ontario, Canada) (*SOSP '19*). Association for Computing Machinery, New York, NY, USA, 225–242. <https://doi.org/10.1145/3341301.3359641>
- Luke Nelson, Jacob Van Geffen, Emina Torlak, and Xi Wang. 2020. Specification and verification in the field: Applying formal methods to BPF just-in-time compilers in the Linux kernel. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, USA, 41–61.
- Tobias Nipkow, Markus Wenzel, and Lawrence C. Paulson. 2002. *Isabelle/HOL: a proof assistant for higher-order logic*. Springer-Verlag, Berlin, Heidelberg.
- Scott Owens, Peter Böhm, Francesco Zappa Nardelli, and Peter Sewell. 2011. Lem: a lightweight tool for heavyweight semantics. In *Proceedings of the Second International Conference on Interactive Theorem Proving (Berg en Dal, The Netherlands) (ITP'11)*. Springer-Verlag, Berlin, Heidelberg, 363–369.
- Thibaut Pérami, Thomas Bauereiss, Brian Campbell, Zongyuan Liu, Nils Lauermaun, Alasdair Armstrong, and Peter Sewell. 2026. ArchSem: Reusable Rigorous Semantics of Relaxed Architectures. *Proc. ACM Program. Lang.* 10, POPL, Article 8 (Jan. 2026), 31 pages. <https://doi.org/10.1145/3776650>
- A. Pnueli, M. Siegel, and E. Singerman. 1998. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems*, Bernhard Steffen (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 151–166.
- Swarn Priya, Frédéric Besson, Connor Sughrue, Tim Steenvoorden, Jamie Fulford, Freek Verbeek, and Binoy Ravindran. 2025. BeePL: Correct-by-compilation kernel extensions. arXiv:2507.09883 [cs.PL] <https://arxiv.org/abs/2507.09883> Accessed: 2026-03-14.
- Christopher Pulte, Shaked Flur, Will Deacon, Jon French, Susmit Sarkar, and Peter Sewell. 2017. Simplifying ARM concurrency: multicopy-atomic axiomatic and operational models for ARMv8. *Proc. ACM Program. Lang.* 2, POPL, Article 19 (Dec. 2017), 29 pages. <https://doi.org/10.1145/3158107>
- Silvain Rideau and Xavier Leroy. 2010. Validating Register Allocation and Spilling. In *Compiler Construction*, Rajiv Gupta (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 224–243.
- Andreas Rossberg. 2017. WebAssembly Specification. <https://webassembly.github.io/spec/core/> Accessed: 2026-03-14.
- Bhat Sanjit and Shacham Hovav. 2023. Formal Verification of the Linux Kernel eBPF Verifier Range Analysis. <https://sanjit-bhat.github.io/assets/pdf/ebpf-verifier-range-analysis22.pdf> Accessed: 2026-03-14.
- Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Gilles Peskine, Thomas Ridge, Susmit Sarkar, and Rok Strniša. 2007. Ott: effective tool support for the working semanticist. *SIGPLAN Not.* 42, 9 (Oct. 2007), 1–12. <https://doi.org/10.1145/1291220.1291155>
- Ben Simmer, Alasdair Armstrong, Jean Pichon-Pharabod, Christopher Pulte, Richard Grisenthwaite, and Peter Sewell. 2022. Relaxed virtual memory in Armv8-A. In *Programming Languages and Systems*, Ilya Sergey (Ed.). Springer International Publishing, Cham, 143–173.
- Solana-labs. 2018. solana rbp. <https://github.com/solana-labs/rbp> Accessed: 2026-03-14.
- Hao Sun and Zhendong Su. 2024. Validating the eBPF verifier via state embedding. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation (Santa Clara, CA, USA) (OSDI'24)*. USENIX Association, USA, Article 33, 14 pages.
- Hao Sun and Zhendong Su. 2025. Prove It to the Kernel: Precise Extension Analysis via Proof-Guided Abstraction Refinement. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) (*SOSP '25*). Association for Computing Machinery, New York, NY, USA, 736–751. <https://doi.org/10.1145/3731569.3764796>
- Hao Sun, Yiru Xu, Jianzhong Liu, Yuheng Shen, Nan Guan, and Yu Jiang. 2024. Finding Correctness Bugs in eBPF Verifier with Structured and Sanitized Program. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) (*EuroSys '24*). Association for Computing Machinery, New York, NY, USA, 689–703. <https://doi.org/10.1145/3627703.3629562>
- Dave Thaler. 2024. BPF Instruction Set Architecture (ISA) draft-ietf-bpf-isa-04. <https://datatracker.ietf.org/doc/draft-ietf-bpf-isa/> Accessed: 2026-03-14.
- Emina Torlak and Rastislav Bodik. 2013. Growing solver-aided languages with rosette. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software* (Indianapolis, Indiana, USA) (*Onward! 2013*). Association for Computing Machinery, New York, NY, USA, 135–152. <https://doi.org/10.1145/2509578.2509586>
- Jacob Van Geffen, Luke Nelson, Isil Dillig, Xi Wang, and Emina Torlak. 2020. Synthesizing JIT Compilers for In-Kernel DSLs. In *Computer Aided Verification*, Shuvendu K. Lahiri and Chao Wang (Eds.). Springer International Publishing, Cham, 564–586.
- Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2022. Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, Virtual Event Republic of Korea, 254–265. <https://doi.org/10.1109/CGO53902.2022.9741267>

- Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2023. Verifying the Verifier: eBPF Range Analysis Verification. In *Computer Aided Verification*, Constantin Enea and Akash Lal (Eds.). Springer Nature Switzerland, Cham, 226–251.
- Jaroslav Ševčík, Viktor Vafeiadis, Francesco Zappa Nardelli, Suresh Jagannathan, and Peter Sewell. 2013. CompCertTSO: A Verified Compiler for Relaxed-Memory Concurrency. *J. ACM* 60, 3, Article 22 (June 2013), 50 pages. <https://doi.org/10.1145/2487241.2487248>
- Xi Wang, David Lazar, Nickolai Zeldovich, Adam Chlipala, and Zachary Tatlock. 2014. Jitk: A Trustworthy In-Kernel Interpreter Infrastructure. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 33–47.
- Xiwei Wu, Yueyang Feng, Tianyi Huang, Xiaoyang Lu, Shengkai Lin, Lihan Xie, Shizhen Zhao, and Qinxiang Cao. 2025. VEP: a two-stage verification tool chain for full eBPF programmability. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (Philadelphia, PA, USA) (NSDI '25)*. USENIX Association, USA, Article 16, 23 pages.
- Qiongwen Xu, Michael D. Wong, Tanvi Wagle, Srinivas Narayana, and Anirudh Sivaraman. 2021. Synthesizing safe and efficient kernel extensions for packet processing. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference (Virtual Event, USA) (SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 50–64. <https://doi.org/10.1145/3452296.3472929>
- Shenghao Yuan, Frédéric Besson, and Jean-Pierre Talpin. 2024. End-to-End Mechanized Proof of a JIT-Accelerated eBPF Virtual Machine for IoT. In *Computer Aided Verification*, Arie Gurfinkel and Vijay Ganesh (Eds.). Springer Nature Switzerland, Cham, 325–347.
- Shenghao Yuan, Frédéric Besson, Jean-Pierre Talpin, Samuel Hym, Koen Zandberg, and Emmanuel Baccelli. 2022. End-to-End Mechanized Proof of an eBPF Virtual Machine for Micro-controllers. In *Computer Aided Verification*, Sharon Shoham and Yakir Vizel (Eds.). Springer International Publishing, Cham, 293–316.
- Shenghao Yuan, Zhuoruo Zhang, Jiayi Lu, David Sanan, Rui Chang, and Yongwang Zhao. 2025. A Complete Formal Semantics of eBPF Instruction Set Architecture for Solana. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 80 (April 2025), 27 pages. <https://doi.org/10.1145/3720414>
- Koen Zandberg and Emmanuel Baccelli. 2020. Minimal Virtual Machines on IoT Microcontrollers: The Case of Berkeley Packet Filters with rBPF. In *PEMWN*. IEEE, Berlin / Virtual, Germany, 1–6.
- Koen Zandberg, Emmanuel Baccelli, Shenghao Yuan, Frédéric Besson, and Jean-Pierre Talpin. 2022. Femto-Containers: Lightweight Virtualization and Fault Isolation for Small Software Functions on Low-Power IoT Microcontrollers. In *Proceedings of the 23rd ACM/IFIP International Middleware Conference (Quebec, QC, Canada) (Middleware '22)*. Association for Computing Machinery, New York, NY, USA, 161–173. <https://doi.org/10.1145/3528535.3565242>